

RAPPORT

Gruppenavn og kandidater:

Gruppe 5

Simen Emil Wiig Holmen (seholmen@uia.no)

Jens Haakaas (jensnh@uia.no)

Lars Kydland (larsky@uia.no)

Siri Johansen Thunestvedt (sirijt@uia.no)

Lena Gjøre (lenagj@uia.no)

Ingri Drabløs Bolstad (ingrib@uia.no)

Emnekode	IS-200
Emnenavn	Systemanalyse og systemutvikling
Emneansvarlig	Geir I. Hausvik Sofie Wass
Innleveringsfrist	20.11.2024
Antall ord (hele dokumentet)	12 744
Tittel på prosjekt/produkt	Sluttrapport

Vi bekrefter at vi ikke siterer eller på annen måte bruker andres arbeid uten at dette er oppgitt og at alle referanser er oppgitt i litteraturlisten.	Ja X	Nei
--	-----------------------	------------

Kan besvarelsen brukes til undervisningsformål?	Ja X	Nei
---	-----------------------	------------

Vi bekrefter at alle i gruppa har bidratt til besvarelsen	Ja X	Nei
---	-----------------------	------------

Innholdsfortegnelse

1. Bakgrunn.....	4
2. Mål.....	5
3. Analyse.....	5
3.1 Systemdefinisjon.....	5
3.1.1 Rich Picture.....	6
3.1.2 FACTOR.....	7
3.2 Analyse av problem domenet.....	8
3.2.1 Problem-domenet.....	8
3.2.2 Klasser.....	8
3.2.3 Struktur.....	9
3.2.4 Objekt struktur.....	10
3.2.5 Klassestruktur.....	11
3.2.6 Adferd.....	11
3.3 Analyse av applikasjonsdomenet.....	12
3.3.1 Bruk av system.....	12
3.3.2 Use case diagram.....	12
3.3.3 Statechart diagram.....	13
3.3.4 Brukerhistorier.....	14
4. Design.....	17
4.1 Arkitekturmodell.....	17
4.2 Kriterier for arkitektur.....	18
4.3 PACT.....	20
4.4 Kriterier for brukergrensesnitt.....	20
4.5 Prioritering av design kriterier.....	21
4.5.1 Prioritering av kriterier for arkitektur.....	22
4.5.2 Prioriteringer av kriterier for brukergrensesnitt.....	23
4.6 Sketching.....	24
4.7 Wireframe.....	25
4.8 Navigation map.....	26
4.9 Prototype.....	28
5. Scrum.....	30
5.1 Hvorfor velge scrum?.....	31
5.2 Pre-Sprint: 26.08.24 - 08.09.24.....	33
5.3 Sprint 1: 09.09.24 - 22.09.24.....	34
5.3.1 Sprint Planning.....	34
5.3.2 Daily Scrum og arbeidet.....	35
5.3.3 Sprint Review og Sprint Retrospective.....	35
5.4 Sprint 2: 23.09.24 - 06.10.24.....	36
5.4.1 Sprint planning.....	36
5.4.2 Daily Scrum og arbeid.....	36
5.4.3 Sprint Review og Sprint Retrospective.....	37
5.5 Sprint 3: 07.10.24 - 20.10.24.....	37

5.5.1 Sprint Planning.....	38
5.5.2 Daily Scrum og arbeidet.....	38
5.5.3 Sprint review og Sprint Retrospective.....	38
5.6 Sprint 4: 21.10.24 - 03.11.24.....	39
5.6.1 Sprint planning.....	39
5.6.2 Daily Scrum og arbeid.....	39
5.6.3 Sprint Review og Sprint Retrospective.....	39
5.7 Sprint 5: 04.11.24 - 17.11.24.....	40
5.7.1 Sprint planning.....	40
5.7.2 Daily Scrum og arbeid.....	40
5.7.3 Sprint Review og Sprint Retrospective.....	41
5.7.4 Refleksjon av Scrum.....	41
5.8 Etikk og universell utforming.....	41
6. Evaluering.....	42
6.1 Analyse.....	43
6.2 Design.....	43
6.3 Scrum.....	44
7. Referanse.....	45
8. Vedlegg.....	47
8.1 Vedlegg 1:.....	47
8.2 Vedlegg 2:.....	47

Figurliste

Figur 1: Rich picture.....	7
Figur 2: Klassediagram.....	10
Figur 3: Use case diagram.....	13
Figur 4: Statechart diagram.....	14
Figur 5: MVC arkitektur.....	18
Figur 6: Fra Prototypen.....	24
Figur 7: Fellessketchen.....	25
Figur 8: Wireframe for "Alle innmeldinger" siden for en Saksbehandler.....	26
Figur 9: Navigation map.....	27
Figur 10: Prototypens hjemmeside.....	28
Figur 11: Prototypens side for "mine innmeldinger".....	29
Figur 12: Prototypens side for innmeldingen av Gjesdal kommune.....	30
Figur 13: Kanbanboard for sprint 1.....	34
Figur 14: Kanbanboard for sprint 2.....	36
Figur 15: Kanbanboard for sprint 3.....	38
Figur 16: Kanbanboard for sprint 4.....	39
Figur 17: Kanbanboard for sprint 5.....	40

Tabell liste

Tabell 1: Factor kriterier.....	8
Tabell 2: Event table.....	9
Tabell 3: Brukerhistorie 1.....	15
Tabell 4: Brukerhistorie 2.....	16
Tabell 5: Brukerhistorie 3.....	16
Tabell 6: Brukerhistorie 4.....	17
Tabell 7: Arkitektur kriterier.....	19
Tabell 8: Brukergrensesnitt kriterier.....	21
Tabell 9: Prioriteringer av arkitektur kriterier.....	22
Tabell 10: Prioriteringer av brukergrensesnitt kriterier.....	23
Tabell 11: Rollene i SCRUM.....	32
Tabell 12: Hendelser og artefakter i SCRUM.....	33

1. Bakgrunn

Kartverket ble etablert i 1773, og er den eldste tekniske etaten som fremdeles er i drift i Norge. Virksomheten er moderne og innovativ, med et blikk for fremtiden. Hovedformålet er å samle inn, systematisere, forvalte og videreformidle offentlig geografisk informasjon. For å holde den geografiske informasjonen oppdatert, har Kartverket en egen funksjon der befolkningen kan melde ifra dersom en opplever at kartet inneholder feil eller mangler. Hvem som helst kan registrere disse feilene og manglene, og funksjonen heter: “rett i kartet” (Kartverket, 2024).

2. Mål

Ved å velge funksjonen: “rett i kartet”, kommer det opp tre mulige alternativer: “rett i turkart og stedsnavn”, “rett i veger og adresser”, og “meld inn turruter”. Uavhengig av hvilket alternativ som blir valgt, kommer man videre til et kart. Her markeres området som trenger oppdatert informasjon. Ved å trykke på: “neste”, er det mulig å beskrive innmeldingen, og deretter sende inn den registrerte endringen. Kartverket har kommet med et ønske om å forbedre denne prosessen. Formålet med oppgaven er derfor å utvikle en løsning som kan hjelpe Kartverket med å oppdatere og forbedre kartgrunnlaget, ved hjelp av en Crowd sourcing-løsning for å få informasjon fra brukere. Et annet ønske, er forbedring av verktøy for saksbehandlere i Kartverket. Ved å innføre et nytt og brukervennlig system, kan Kartverket oppnå sine ønsker om et enkelt og effektivt verktøy, som vil resultere i en enklere prosess både for saksbehandler og innbygger.

Målet med rapporten er å fremstille hvordan gruppen har utført prosjektet gitt av Kartverket i samarbeid med Universitetet i Agder, med fokus på analyse og design. Arbeidet er organisert ved bruk av prosjektrammeverket Scrum, og Kanbanboardet Trello.

Underveis i utviklingsfasen hadde gruppen Q&A med saksbehandlere fra Kartverket. På grunn av dette, dukket det stadig opp ny informasjon, og nye ønsker fra Kartverkets side. Dette førte til at gruppen måtte gjøre regelmessige endringer og tilpasninger i webapplikasjonen. Ettersom gruppen valgte å benytte seg av Scrum, har det vært mulig å imøtekomme disse endringene på en fleksibel og effektiv måte. Rapporten inneholder dermed de endelige funksjonene og løsningene i gruppens webapplikasjon.

3. Analyse

3.1 Systemdefinisjon

For å forstå hvordan løsningen skal utvikles, er det nødvendig med en systemdefinisjon. En systemdefinisjon er en presis beskrivelse av et digitalt system utformet i naturlig språk. Det er

en måte å skaffe oversikt, og sammenlikne ulike perspektiver og alternativer. Dette er en viktig del av designprosessen, ettersom det gir en klar retning for hvordan systemet skal være, basert på ønsker fra Kartverket. Det forsikrer at alle gruppe medlemmene jobber mot et felles mål (Wass, S. Systemanalyse og systemutvikling, 2024).

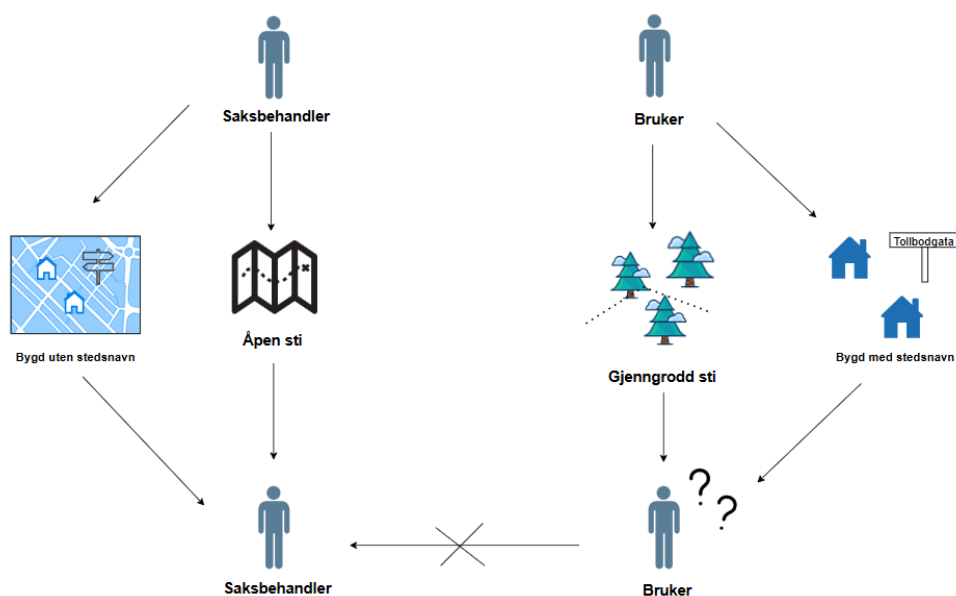
Prosjektet har opphav fra Kartverket sitt eksisterende system, der de ønsker å gjøre det lettere å melde inn endringer (se vedlegg 1). Innmeldingene kommer i form av oppdateringer av manglende informasjon i kart, for eksempel knyttet til turstier, veier og stedsnavn. I prosjektbeskrivelsen har Kartverket identifisert og beskrevet problemstillingen. Her har det kommet frem ønske om at applikasjonen skal utvikle en løsning som kan hjelpe dem med å oppdatere og forbedre kartgrunnlaget ved hjelp av informasjon fra brukerne. Kartverket ønsker derfor en enkel Crowd sourcing-løsning som kan benyttes av hele befolkningen. Systemet må være brukervennlig, og lett tilgjengelig. Et annet ønske, er forbedring av verktøy for saksbehandlere i Kartverket, for å kunne behandle innmeldinger, og tildele oppgaven videre til andre saksbehandlere.

Systemdefinisjonen må dekke det prosjektbeskrivelsen legger frem. For å tydeliggjøre og utforme en god systemdefinisjon, har gruppen tatt i bruk Rich Picture og FACTOR.

3.1.1 Rich Picture

Rich pictures er en illustrasjon som skal tydeliggjøre en situasjon. Poenget er å reflektere over konflikter, prosesser, strukturer, og mennesker involvert. Denne metoden er en måte å utforske, bekrefte og definere en situasjon. Dette kan bidra til videre diskusjon og refleksjon rundt temaet. Dette kan gjøre det lettere å komme frem til en bred og felles forståelse av den nåværende prosessen, og de ulike faktorene dette innebærer (Stevens, u.d.).

Gruppens rich picture er demonstrert i figur 1. Venstre side fremstiller en saksbehandler i Kartverket. Den ansatte ser på kartet, og finner frem en bygd uten et loggført stedsnavn. Ifølge kartet har også bygda en liten tursti, med åpent terreng i nærområdet, som dermed er oversiktlig og lett tilgjengelig. Høyre side fremstiller en turgåer som kommer over denne bygda i virkeligheten. Turgåeren finner fort ut hva bygda heter, og går inn på kartet for å finne et fint turområde. Navnet på bygda er som sagt ikke registrert i kartet, noe som gjør det utfordrende å finne frem. Stien blir funnet til slutt, men den er overgrodd, og umulig å følge. Turgåeren lurer videre på hvordan man skal gå frem for å melde inn disse endringene.



Figur 1: Rich picture

3.1.2 FACTOR

FACTOR er et rammeverk som brukes for å utvikle og klargjøre en system-definisjon. Dette rammeverket består av seks elementer: "functionality", "application domain", "conditions", "technology", "objects" og "responsibility" (Mathiassen et al., s. 40).

Ved å definere disse kategoriene, blir systemets funksjonalitet tydeliggjort. Dette gjør også systemet mer oversiktlig, noe som gjør det lettere å forstå hva som skal være med, og hvem som skal bruke systemet. Tabell 1 viser gruppens FACTOR definisjon.

FACTOR Criteria	Explanation
Functionality	Et system for å melde inn, behandle og se status for endringer i terrenget, som ikke er gjenspeilet i kartet.
Application domain	Systemet vil bli benyttet av saksbehandlere i kartverket og alle personer som ønsker å melde fra om avvik i kartet.
Conditions	Brukere vil ha forskjellige nivåer av IT-kunnskaper.
Technology	A responsive web application.
Objects	Bruker, innmelding, kommune, kart, status, saksbehandlere

Responsibility	En brukervennlig løsning for at Kartverket skal kunne ta imot og behandle endringer, som er registrert av brukerne.
----------------	---

Tabell 1: Factor kriterier

Basert på FACTOR-elementene og rich picture ble gruppens systemdefinisjon slik:

En responsiv web-applikasjon som forenkler innmelding, behandling og status-oppfølgning av endringer i terrenget som ikke er reflektert i det nåværende kartet. Det skal være intuitivt og simpelt, for å gjøre det enklere for brukere med varierende nivåer av IT-kunnskap å melde inn nye terrengendringer. Brukere av systemet vil benytte det til å melde inn noe de ser i terrenget, som avviker fra kartet. Saksbehandlere ved Kartverket vil bruke systemet til å administrere og kontrollere innmeldte endringer, samt følge opp disse sakene over tid. Systemet er utviklet for å være en enkel og brukervennlig løsning for både profesjonelle saksbehandlere og vanlige brukere som ønsker å bidra til oppdatering av kartdata. Det har som hovedmål å sikre en smidig prosess for behandling av kartoppdateringer og å forbedre samarbeidet mellom innbyggere og Kartverket.

3.2 Analyse av problem domenet

3.2.1 Problem-domenet

Problem-domenet defineres som delen av en kontekst som overvåkes, administreres eller kontrolleres av et system (Mathiassen et al., 2018, s.6). Det å forstå og definere problem-domenet er en svært viktig del av prosjektutviklingen. Ved å ha en dypere forståelse av konteksten, er det lettere å klargjøre potensielle utfordringer og risikoer (Institute of data, 2023). For å kunne definere prosjektdomenet, har gruppen tatt utgangspunkt i tre hovedtema: “klasser”, “objekter” og “adferd” (Mathiassen et al., 2018, s. 48-50).

3.2.2 Klasser

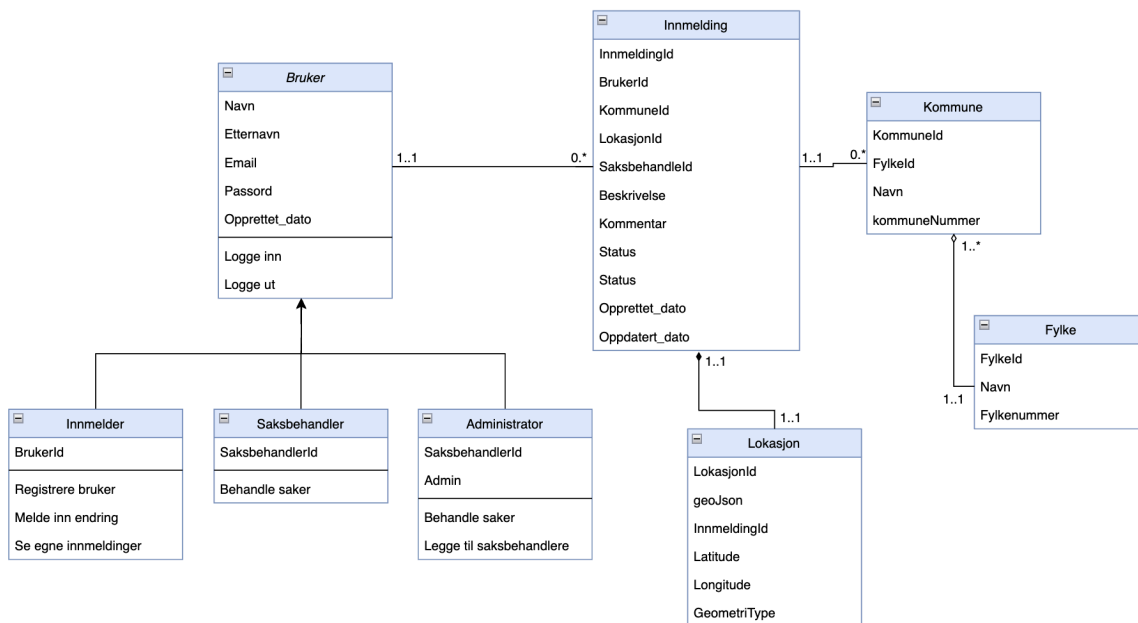
Basert på de tidligere identifiserte prosessene, kunne gruppen definere alle de involverte objektene. Ut fra dette, kunne gruppen videre lage klasser med ulike egenskaper. (Mathiassen et al., 2018, s.51). For å få en oversikt over de mest essensielle klassene i prosjektet, og de tilknyttede hendelsene, er det mulig å lage et Event-table (tabell 2). Tabellen fungerer som et verktøy som analyserer og beskriver ulike hendelser, hvordan de utføres, og hvilke klasser som er involvert. Handlingene står skrevet vertikalt, mens klassene er skrevet horisontalt (Mathiassen et al., s. 51-52).

Events	Klasser		
	Innmelder	Saksbehandlere	Administrator
Reg. endring	*		
Reg. brukere	x		*
Logge inn	*	*	*
Logge av	*	*	*
Sjekke status	*		
Godkjenne/Avvise innmeldinger fra innmeldere		*	*
Sende tilbakemeldinger til innmeldere		*	*
Tildel roller (saksbehandler eller administrator)			*
Gi saker videre til andre saksbehandlere		*	*
+ Kan skje null eller en gang			
* Kan skje null eller flere ganger			

Tabell 2: Event table

3.2.3 Struktur

Videre i prosjektets problem-domene, må det dannes struktur for klassene og objektene som er definert i event-modellen (Mathiassen et al., 2018, s. 70-71). Gruppen dannet derfor et klassediagram (figur 2). Et klassediagram er et digitalt verktøy som presenterer en systemstruktur ved å vise klasser, attributter, metoder, og forholdet mellom dem. Diagrammet gjør det enklere for alle involverte i prosjektet å forstå hvordan systemet er organisert, og dermed kartlegge de ulike klassene, slik at det er enklere å forstå hvordan de ulike komponentene samhandler (Geeksforgeeks, 2024).



Figur 2: Klassediagram

3.2.4 Objekt struktur

Strukturanalysen for objektorientert struktur omfatter de utformede relasjonene mellom objekter, ut fra strukturene: “aggregasjon” og “assosiasjon” (Mathiassen et al., 2018, s. 77).

En aggregasjon struktur betyr at et objekt er en nødvendig og betydelig del av et eller flere andre objekter (Mathiassen et al., 2018, s. 77). Figur 2 demonstrerer aggregasjon som en hvit diamant. Klassen “fylke” er en betydelig del av klassen “kommune”, og er avhengig av “fylke”, fordi den trenger “fylkeId”. Selv om det er en betydelig del, må det ikke eksistere en instans av “kommune” for at fylke skal eksistere.

Den svarte diamanten derimot, viser en sammensetning mellom “lokasjon” og “innmelding”, som betyr at “lokasjon” er avhengig av “innmelding” (Parker, 2024). Dette vil si at en lokasjon ikke kan eksistere uten en innmelding, på grunn av at den eneste måten å få en lokasjon på er å finne ut hvor innmeldingen er meldt inn.

Akkurat som med aggregasjons strukturen, er også en assosiasjon struktur en relasjon mellom to eller flere objekter. Denne strukturen brukes når et objekt har en relasjon til et eller flere andre, men ikke avhenger av hverandre slik som i aggregasjons strukturen. I figur 2 demonstreres blant annet en assosiasjonsstruktur mellom “bruker” og “innmelding”. Selv om det blir utført en endring i “innmelding”, vil ikke “bruker” bli påvirket. Dette er fordi det kun er en relasjon mellom dem, og ikke en avhengighet. Dette vil si at de ulike objektene kan eksistere uten hverandre, men de har likevel noe med hverandre å gjøre (Mathiassen et al., 2018, s. 78-79).

3.2.5 Klassestruktur

Klassestrukturen kan også deles inn i to: “generalisert” og “klynge” (Mathiassen et al., 2018, 79).

I en generalisert klassestruktur representerer overordnede klasser, som refereres til som superklasser, og tilhørende subklasser som spesialiserer seg etter superklassen. Subklassene arver funksjoner, egenskaper og attributter fra den overordnede superklassen, noe som muliggjør gjenbruk av egenskaper. Dette skaper en hierarkisk struktur i systemet (Mathiassen et al., 2018, s. 80). I figur 2, er det demonstrert at “innmelder”, “saksbehandler” og “administrator” arver fra klassen “bruker”. Dette vil si at “innmelder”, “saksbehandler” og “administrator” er subklasser og “bruker” er en superklasse.

Klassestrukturen trenger ikke nødvendigvis kun å bestå av klasser som arver av hverandre. Det er også mulig å gruppere assosierte klasser i klynger. Hver klynge trenger ikke nødvendigvis å ha en konkret relasjon, men heller en generell forbindelse som gjelder for flere klasser samtidig (Mathiassen et al., 2018, s. 81-82). I klassediagrammet (figur 2) er for eksempel klassene: “kommune”, “fylke” og “lokasjon”, en klynge som kan kalles “lokalisering”, på grunn av at klassene forteller alle om hvor på kartet endringen befinner seg. De ulike klassene kan alle være en del av prosessen for å melde inn en endring for en bruker.

3.2.6 Adferd

Adferd er den siste delen av analysen til problem-domenet. Hensikten med denne delen, er å skape en oversikt og forståelse for de ulike hendelsene en klasse kan utføre (Mathiassen, et al., 2018, s. 91).

Tabell 2 viser de ulike hendelsene klassene i gruppens system kan utføre, og hvilken rekkefølge dette skjer i.

- Alle klasser (innmelder, saksbehandler, administrator) kan logge inn flere ganger
- Alle klasser (innmelder, saksbehandler, administrator) kan logge ut flere ganger
- Innmelder kan registrere endringer flere ganger
- Innmelder og administrator kan registrere bruker en gang
- Innmelder kan sjekke status flere ganger
- Saksbehandlere og administrator kan godkjenne og avvise innmeldinger fra innmeldere flere ganger
- Saksbehandlere og administrator kan sende tilbakemelding til innmeldere flere ganger
- Administrator kan tildele roller som saksbehandler eller administrator flere ganger
- Saksbehandlere og administrator kan gi saker videre til andre saksbehandlere flere ganger

3.3 Analyse av applikasjonsdomenet

Applikasjonsdomenet omfatter organisasjonen som overvåker, administrerer eller kontrollerer et problem-domenet (Mathiassen et al., 2018, s. 6). Dette omfatter hvordan systemet håndterer og styrer de ulike aspektene beskrevet i problem-domenet. Formålet med analysen av applikasjonsdomenet er å forstå og finne ut av hvordan systemet vil bli brukt (Mathiassen et al., 2018, s. 117). I prosjektbeskrivelsen kommer det frem at systemet skal gjøre det lettere for brukere å melde inn endringer ved manglende informasjon i kart. Det er derfor viktig at systemet er brukervennlig, og enkelt. Det er også viktig at saksbehandlere får lett tilgang til systemet for å kunne se alle innmeldingene Kartverket mottar. Det må være mulig å se status på innmeldingene, både for saksbehandler og innmelder, og det er derfor viktig at systemet er oversiktlig og lett å orientere seg rundt (se vedlegg 1).

Funksjoner brukeren har behov for i applikasjonsdomenet:

- Registrere bruker/ logge inn
- Registrere innmeldinger
- Ha oversikt over status
- Legge inn endringer
- Godkjenne/ avvise innmeldinger
- Gi saker videre til andre saksbehandlere
- Sende tilbakemelding til innmelder
- Tildele roller

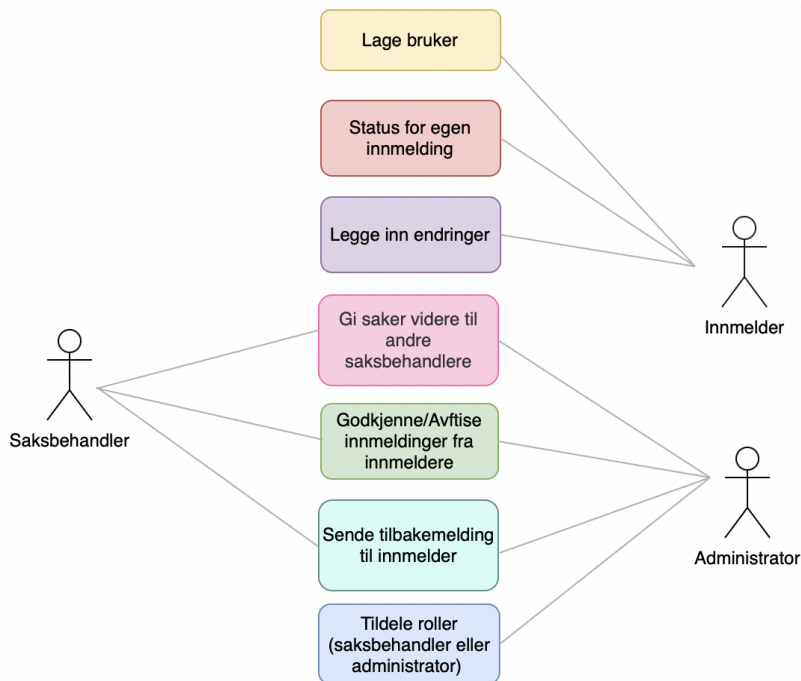
3.3.1 Bruk av system

For at systemet gruppen lager skal være brukbart, er det viktig å fokusere på hvordan brukere vil samhandle med systemet (Mathiassen et al., 2018, s. 121). For å lettere forstå applikasjonsdomenet og tydeliggjøre hvilken kontekst systemet vil bli brukt i, har gruppen tatt for seg use case diagram, statechart diagram, og ulike brukerhistorier.

3.3.2 Use case diagram

Et use case diagram viser interaksjonen mellom systemet og brukere i applikasjonsdomenet. Det er et grafisk illustrert diagram, som viser hvordan systemet vil bli brukt av de ulike partene. En av fordelene med å bruke use case diagram, kan være at man får en enkel forståelse over hvordan systemets oppførsel vil være. Det kan også føre til bedre kommunikasjon og samarbeid mellom utviklere og produkteiere. Diagrammet demonstrerer også helt enkelt hvordan flyten mellom hver handling vil være, samtidig som at kontekst og krav for systemet blir spesifisert (Lucidchart, u.d.).

Figur 3 er et use case diagram for gruppens system, der gruppemedlemmene har valgt de mest relevante funksjonene for systemet, basert på kravene til Kartverket (se vedlegg 1). Use case diagrammet bidro til at gruppen fikk en felles illustrert forståelse av hvordan interaksjonen med systemet vil foregå. Den har også gjort at funksjonene ble mer oversiktlige og gitt gruppen en bedre forståelse av hvordan funksjonene henger sammen.

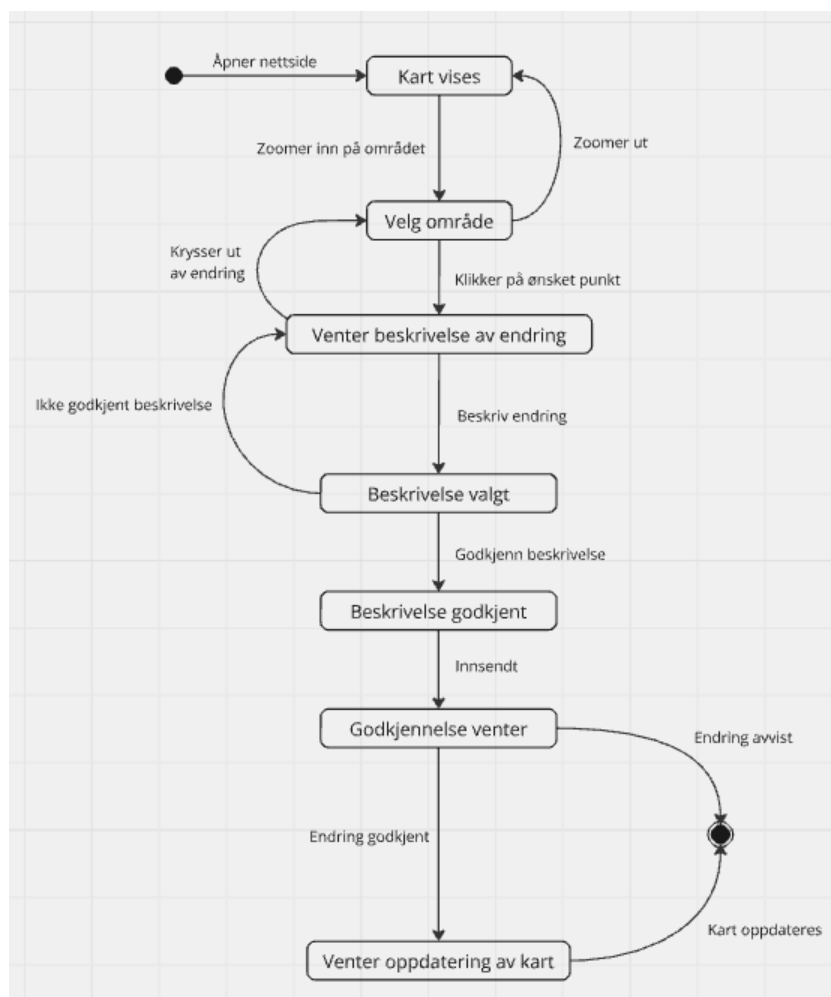


Figur 3: Use case diagram

3.3.3 Statechart diagram

Basert på use case diagrammet, er det mulig å lage et statechart. Et statechart er et diagram som demonstrerer den dynamiske oppførselen og aktiviteten i applikasjonsdomenet. “States” (statuser) og “events” (hendelser) blir brukt for å vise en gjennomgang av hvordan systemet skal oppføre seg ved forskjellige interaksjoner, og hvordan den sekvensielle flyten i systemet er (Lewis, 2023).

Startpunktet demonstreres ved hjelp av en svart prikk, som vist i figur 4. Den optimale flyten av events for en bruker av systemet er som følger: åpner nettside > zoomer inn på området > klikker på ønsket punkt > beskriver endring > beskrivelse godkjennes > sendes inn > endring godkjennes > kart oppdateres.



Figur 4: Statechart diagram

3.3.4 Brukerhistorier

Brukerhistorier er enkle beskrivelser av en alternativ funksjonalitet til et programvareprodukt. Hensikten med dette verktøyet er å utforske mulige funksjonaliteter, og tydeliggjøre kommunikasjon mellom utviklere og designere (Mikalsen, 2024). Gruppen har laget flere ulike brukerhistorier basert på de systemspesifikasjonene gitt av Kartverket (Vedlegg 1). Dette bidrar til en bedre oversikt og en felles forståelse av hvordan det fremtidige systemet vil fungere.

For å angi prioriteringer til brukerhistoriene, har gruppen utnyttet MoSCoW-metoden. Dette er en teknikk for å prioritere viktigheten av oppgaver, og den deles inn i fire kategorier: "must have", "should have", "could have", og "won't have" (Matyska, 2022). Metoden har gjort det enklere å definere hvilke funksjoner som er nødvendige å ha med, og hvilke som er mindre viktige. Som vist i tabell 3-6, inneholder brukerhistoriene argumenter for hvorfor funksjonene er prioritert slik.

I denne oppgaven ble det laget 12 ulike brukerhistorier. Det er benyttet to ulike formater for brukerhistoriene, ettersom dette gjør dem mer fleksible og skaper større variasjon:

1. Format #1: "som en `rolle/stilling` vil jeg `handling` får å `motivasjon`".
2. Format #2: "For å `handling` vil jeg som `rolle/stilling` ha mulighet til å `handling`".

(Wass, S. Systemanalyse og systemutvikling, 2024).

#1	Prioritet: Must have	Format 1
Brukerhistorie	Som en bruker ønsker jeg å kunne opprette en brukerkonto slik at jeg har oversikt og trygghet på Kartverket.no	
Funksjon	Opprette en bruker på Kartverket	
Kriterier	Man kan opprette en bruker, slik at det blir god oversikt for brukeren å rapportere inn endringer. Må opprette et brukernavn og passord	
Test-scenario	1. Trykk på "Opprett en bruker" 2. Skriv inn ønsket brukernavn 3. Lag et passord 4. Logg inn	
Argumentasjon	Å lage en bruker er en veldig essensiell funksjon for systemet og er derfor plassert som en "must have". Det gjør at brukeren får oversikt over sine rapporterte endringer. Dette kan også bidra positivt for saksbehandlere, ettersom innmeldingene vil være mer seriøse når innmeldere har opprettet bruker.	

Tabell 3: Brukerhistorie 1

#2	Prioritet: Should have	Format 1
Brukerhistorie	Som en bruker ønsker jeg å kunne legge inn bilder av endringer som er gjort i et område, slik at det er tydeliggjort hvilket område jeg refererer til.	
Funksjon	Legge inn bilde av endringer	
Kriterier	Endring i naturen skjer stadig vekk og derfor er det viktig å dokumentere og dele disse endringene for god kartoversikt.	
Test-scenario	1. Logg inn 2. Finn siden som heter "Nytt innlegg" 3. Velg ønsket bilde eller ta et bilde ved å trykke på knappen. 4. Last opp innlegget.	

Argumentasjon	Det er viktig å ha mulighet for å legge inn bilder av endringer for å holde kartet oppdatert. Dette skaper en bedre brukeropplevelse for individene som bruker kartet og tydeliggjør hvilket område som blir referert til. Funksjonen er lagt til “should have” fordi den er en viktig del av systemet.
---------------	---

Tabell 4: Brukerhistorie 2

#10	Prioritet: Could have	Format 2
Brukerhistorie	For å ha oversikt og trygghet ønsker jeg å bruke min e-post når jeg skal lage en bruker hos Kartverket.	
Funksjon	Bruke e-post for å lage bruker	
Kriterier	Å få informasjon på en plass gjør det lettere å holde seg oppdatert på endringer gjort av deg selv eller andre	
Test-scenario	1. Trykk på “Sign up” 2. Bruk e-post for å lage bruker 3. Lag passord 4. Trykk på “fullfør bruker”	
Argumentasjon	Dersom innmeldere har egen bruker koblet til e-post, er det enkelt å få en oversikt over egne innmeldinger, og se statuset. Dette gjør det også mulig å komme med en oppfølging av tidligere innmeldinger. Denne funksjonen er plassert på “could have” ettersom det ikke er et krav fra oppgave bestillingen.	

Tabell 5: Brukerhistorie 3

#12	Prioritet: Won't have	Format 2
Brukerhistorie	For å få en forklaring i hvordan man bruker systemet, som en turgåer, ønsker jeg å ha kundeservice som kan forklare meg hvordan jeg skal bruke løsningen.	
Funksjon	Ha en tekst-chat, som brukere kan kommunisere med kundeservice.	
Kriterier	Brukere skal ha mulighet til å kontakte kundeservice enkelt og få relativt raskt svar på det de lurte på i forbindelse med systemet.	
Test-scenario	1. Logge inn som bruker 2. Klikke på chat-knapp nede i høyre hjørne 3. Skrive et spørsmål og trykke på send 4. Få svar i samme chat av kundeservice	

Argumentasjon	Kundeservice er noe som nesten er forventet å ha i enhver digital løsning. Brukere har ofte spørsmål, som kundeservice enkelt kan besvare. Dette fører til mindre frustrasjon blant brukerne. Funksjonen er uansett satt til “won’t have”, da det antas at løsningen vil være såpass enkel å forstå, og ha nok informasjon om hvordan man bruker den, til at det ikke skal være nødvendig med tekst-chat.
---------------	---

Tabell 6: Brukerhistorie 4

4. Design

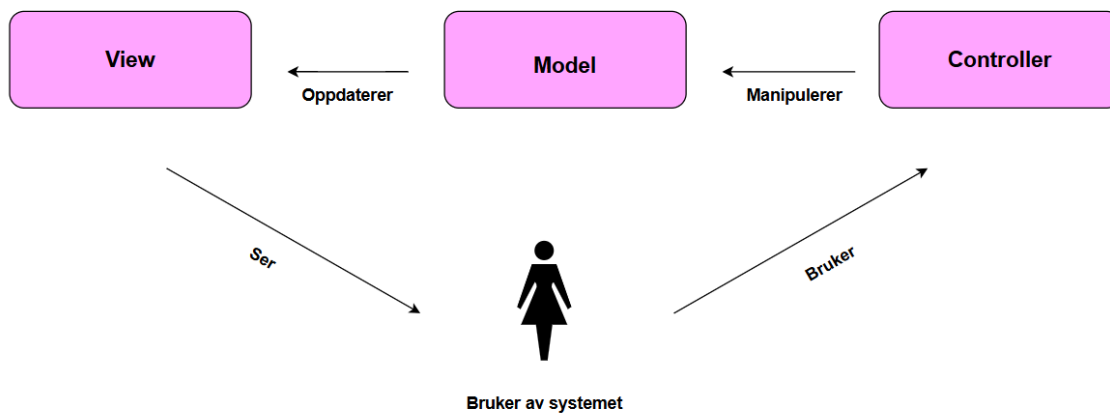
Design handler om hvordan funksjoner i et gitt informasjonssystem blir gjort tilgjengelig for brukere på best mulig måte (Mathiassen et al., 2018, s.153). For å lykkes med dette, finnes det ulike design krav. Struktur er en sentral del av en webapplikasjon. For å skape god struktur i designet, finnes det noen krav til arkitektur som kan og burde vurderes. Disse kravene bidrar til å sikre programvarens kvalitet (Mathiassen et al., 2018, s. 175 & 180). Det er også nødvendig å skape et design som er brukervennlig. Det finnes derfor krav for brukergrensesnitt, som gjør det lettere å skape en enkel og vellykket webapplikasjon (Benyon, 2019, s.117).

4.1 Arkitekturmodell

I dette prosjektet har gruppen utnyttet seg av MVC-arkitektur. Det vil si at systemet har blitt separert i tre ulike deler: “model”, “view” og “controller”. Model- delen håndterer systemets data, view omfatter det brukerne ser, mens controller håndterer bruker input, og forbinder de to andre delene. Denne fordelingen åpner opp for å arbeide på hver del separat, noe som gjør det mulig å gjøre endringer uten å påvirke hele systemet. Det blir dermed mindre krevende å legge til nye egenskaper og holde alt organisert, og det forbedrer også kvaliteten på softwaren (Geeksforgeeks, 2024). “View” omfatter som sagt det brukeren ser, og baseres dermed på designet til webapplikasjonen. Målet er at systemet skal være så brukervennlig som mulig, og inneholde god struktur. For å lykkes med dette, er det nødvendig at designet dekker ulike krav for arkitektur og brukergrensesnitt (Mathiassen et al., 2018, s. 117 & 179).

En implementasjon av MVC-arkitektur kan utføres på mange ulike måter. Hvordan arkitekturen er implementert, avhenger av preferanser, hva som skal utvikles, og hva som passer best til prosjektet. En tilnærming er å konseptualisere MVC der Controller-delen styrer hoveddelen av oppgavene. Controller kontrollerer all brukerinteraksjon, leverer data til view, og styrer kommunikasjonen mellom Model og brukergrensesnittet. Den lineære strukturen er en annen tilnærming av MVC. Her samhandler brukerne kun med viewt via en nettside, viewet kommuniserer med Controller, og Controller viderefører data til Model (Sheldon, 2023). Gruppens versjon av MVC-strukturen ligner mest på den lineære metoden. I tillegg til

de tre klassiske komponentene i MVC, har gruppen introdusert en fjerde komponent, Service. I Service-mappen ligger spørringene til databasen i prosjektet. Mappen tar i bruk `ConnectionString` og det virker som et bindeledd mellom det fullstendige systemet og databasen.



Figur 5: MVC arkitektur

4.2 Kriterier for arkitektur

Det er viktig at systemet fremviser problem-domenet som tidligere er blitt definert, at det inneholder alle funksjonelle krav, og at grensesnittet tilbyr de spesifiserte interaksjonene. Likevel er også design en nødvendig del av applikasjonen. Dersom designet inneholder en avgjørende feil, kan det føre til at systemet blir utfordrende. For å unngå dette, er det lurt å utnytte noen spesifiserte design kriterier. Tabell 7 inneholder alle design kriteriene gruppen har valgt for arkitekturen. Ifølge boken *Object Orientated Analysis & Design*, er dette klassiske kriterier som ofte blir brukt for å sikre programvarens kvalitet (Mathiassen et al., s. 179-180).

Kriterier	Basert på
Usable	Systemets evne til å tilpasse seg den tekniske og arbeidsrelaterte konteksten.
Efficient	Det handler om å utnytte økonomiske delene av de tekniske plattformene.
Reliable	Gjennomføring av de nødvendige funksjonelle utførelsene.
Maintainable	Kostnaden av å finne og fikse feil i systemet
Comprehensible	Innsatsen som trengs for å få en samlet forståelse av systemet

Testable	Kostnaden av å forsikre at systemet fungerer og oppnår ønsket funksjon
Flexible	Å tilpasse til ulike brukerbehov og arbeidsmåter for å sikre at brukere med forskjellige preferanser kan tilpasse seg deres behov.
Interoperable	Evnen til å fungere sammen med andre systemer og plattformer.
Correct	At systemet skal være konsistent slik at brukerne kan stole på at systemet alltid fungerer slik forventet.

Tabell 7: Arkitektur kriterier

“Usable” er et viktig kriterium for arkitektur design, fordi det fokuserer på at systemet har god funksjonalitet og tilpasningsevne. I dette ligger det ikke bare at systemet må fungere i praksis, men også sørge for balanse mellom brukervennlighet og tekniske krav (Mathiassen et al., s. 182). Kriteriet “Efficient” er sentralt for ressursbruk og ytelse. Med effektiv ressursutnyttelse vil systemet ha lavere driftskostnader, samtidig som det håndterer store datamengder. Dette vil gjøre systemet robust. “Reliable” betyr at systemet må være til å stole på. Det fører til at flere brukere vil ta i bruk systemet og de kan være sikre på at ønskede resultat blir levert. Pålitelighet vil lette vedlikehold, ivareta dataintegritet og opprettholde forretningsmålene (Mathiassen et al., s. 180).

“Maintainable” er et viktig kriterium for systemet, fordi det stadig må vedlikeholdes. Ved å holde systemet vedlike, vil det være enklere og mer effektivt å finne og fikse feil. Kravet er derfor basert på kostnaden av å finne og fikse disse feilene i systemet. For å kunne bekrefte at systemet fungerer som det skal, må det også være mulig å gjennomføre tester underveis i utviklingsprosessen. Dette kriteriet er basert på kostnaden av å forsikre at systemet fungerer og oppnår ønsket funksjon. Ved å teste regelmessig, finner man gjerne feil tidlig i utviklingsprosessen, noe som gjør det enklere og billigere å fikse (Mathiassen et al., s. 180). Det er også viktig at systemet er “comprehensible”, altså forståelig, slik at brukerne enkelt skal kunne ta det i bruk. Kriteriet er derfor basert på innsatsen som trengs for å få en samlet forståelse av systemet. Abstraksjon er et viktig verktøy for å lykkes med dette på best mulig vis. Systemet burde fungere på en logisk måte, slik at brukerne skal forstå hvordan det fungerer, uten å måtte legge inn mye tid og innsats (Mathiassen et al., s. 183).

Ulike brukere har ulike preferanser, enten det gjelder personlig tilpasning av funksjoner eller arbeidsflyt. “Flexibility” er derfor et viktig design kriterium. Et fleksibelt brukergrensesnitt gir muligheter for tilpasning, som øker både effektiviteten og brukeropplevelsen. Dette er spesielt viktig der brukerne kan ha svært forskjellige ferdigheter og behov (Mathiassen et al., s. 182). “Interoperable” refererer til brukergrensesnittets evne til å samhandle sømløst med andre systemer og plattformer. I en verden der systemer stadig oftere må fungere mellom ulike enheter og plattformer, er “interoperable” en faktor for å sikre effektivitet. Systemer som er lett å integrere med andre løsninger, muliggjør bedre sammenheng mellom ulike teknologier og prosesser. Til slutt er kriteriet “correct” en grunnleggende egenskap som

omhandler systemets konsistens. Et velfungerende system sørger for at systemet fungerer slik det skal hver gang, uten feil. Dette bygger tillit og bidrar til en mer effektiv prosess, da brukerne vet hva de kan forvente og kan fokusere på sine oppgaver uten å bekymre seg for tekniske problemer (Mathiassen et al., s. 180).

4.3 PACT

For å skape et system med høyest mulig brukervennlighet har gruppen tatt i bruk PACT-analyse. Analysen er med på å skape en helhetlig forståelse av brukeren og deres behov i ulike sammenhenger. PACT er delt inn i fire ulike hovedkategorier: “People”, “Activity”, “Context” og “Technology” (Benyon, 2019, s.31). Ved å ta i bruk denne modellen, har gruppen fått en bedre forståelse for hvem målgruppen er, hvilke mål og oppgaver de har, og hvilken kontekst systemet vil bli brukt i. Det har også gitt en oversikt over hvilke teknologier som er tilgjengelig og forenelig med systemet (Benyon, 2019, s.31-43).

Målgruppen er folk i alle aldre som ønsker å rapportere om endring i ett kart. Det er derfor viktig at systemet er intuitivt og lett å forstå slik at det kan brukes av alle, uavhengig av teknologiske ferdigheter og mental-modeller. Oppgavene i systemet bør derfor organiseres i ett i ett enkelt step-by- step-system, slik at bruken blir enklere. Dette gjør det også lettere å kunne bruke det i en kontekst hvor man f.eks. er ute i skogen, eller om man er på farten. For å sikre best mulig brukervennlighet er det også viktig at systemet både kan brukes på mobil og pc.

4.4 Kriterier for brukergrensesnitt

Brukergrensesnitt er en betegnelse på kontaktflaten mellom systemet og brukeren. Digitale enheter har et fysisk brukergrensesnitt i form av tastatur, knapper, skjerm og display. Disse enhetene inneholder også grafiske brukergrensesnitt. I de fleste tilfeller vil et grafisk brukergrensesnitt kontrolleres via en mus eller en berøringsskjerm, der man peker og klikker på knapper for å styre programmer. Systemer med slike brukergrensesnitt er vanligvis enkle å lære seg (Nätt, 2023). For å skape et velfungerende system, er det også kriterier for brukergrensesnitt. Gruppen har fokusert på Benyons prinsipper, som vist i tabell 8. De valgte prinsippene gjør det lettere å skape et brukergrensesnitt som har høy brukervennlighet (Benyon, 2019, s.117-123). Gruppen valgte derfor 6 prinsipper som samsvarer med resultatet fra PACT analysen.

Kriterier	Basert på
Familiarity	Designet skal være likt som andre design med samme funksjon.

Consistency	Designet må være likt mellom alle de forskjellige sidene.
Navigation	Det må kunne være lett å navigere gjennom designet slik at man finner funksjonene man trenger.
Feedback	Gi hyppige tilbakemeldinger slik at brukeren vet at handlingene deres har en effekt på systemet.
Recovery	Om brukeren har gjort en feil eller fått en error, må en gjøre det lett å komme seg tilbake fra denne feilen/fejlen.
Visibility	At ulike funksjoner og handlinger blir synlige.

Tabell 8: Brukergrensesnitt kriterier

"Familiarity" er et kriterium som handler om at designet på systemet burde være basert på andre liknende systemer. Dette vil føre til at brukeren enkelt vil forstå hva de ulike knappene og ikonene betyr, og det vil være lett å forstå hva som skal gjøres for å lokalisere seg frem i systemet. Et annet kriterium er "consistency", som handler om at designet må være likt mellom de ulike sidene i systemet. Dette er viktig, ettersom stor kontrast i designet vil føre til forvirring hos brukerne. Det kan tolkes som at de har blitt videreført til en annen nettside, noe som gjør dette kriteriet sentralt for gruppen. "Navigation" handler om at det skal være enkelt å navigere seg gjennom designet slik at man finner de ulike funksjonene man trenger. Systemet burde derfor være ryddig og lett tilgjengelig, slik at det er enkelt å forstå hvor de ulike funksjonene befinner seg (Benyon, 2019, s. 117).

"Feedback" er også et viktig kriterium for brukergrensesnittet. Tilbakemeldinger og bekreftelse på interaksjoner i systemet bidrar til bedre forståelse og brukervennlighet. Dette gjør det lettere for enhver bruker, uavhengig av teknologiske ferdigheter, å bruke systemet (Benyon, 2019, s.117-123). Et annet viktig kriterium for brukergrensesnittet, er "recovery". Dette kriteriet gjør det mulig for en bruker å returnere eller rette opp når de trykker feil. Dersom en bruker får god erfaring med "recovery" i et system, vil sannsynligheten for at de gir opp systemet minke. Til slutt valgte gruppen å fokusere på "visibility". Dette kriteriet er viktig, ettersom synligheten på knapper og funksjoner vil avgjøre om brukeren klarer å lokalisere seg frem i systemet (Benyon, 2019, s.117).

4.5 Prioritering av design kriterier

Et av prinsippene for å oppnå et bra design, handler om å kunne balansere flere kriterier samtidig. De ulike design kriteriene kan komme i konflikt med hverandre, og må derfor prioriteres etter viktigheten (Mathiassen et al., 2018, s. 188). Prioriteringen kan hjelpe med å lage en spesifikk strategi for videre utvikling av webapplikasjonen. Planleggingen av videre arbeid blir dermed lettere å konkretisere, og prioriteringen vil også gjøre det lettere å beregne når designprosessen er ferdig (Mathiassen et al., 2018, s. 187). Gruppen har derfor laget

diagrammer for å prioritere de valgte kriteriene for arkitektur (tabell 9) og brukergrensesnitt (tabell 10).

4.5.1 Prioritering av kriterier for arkitektur

Kriterier	Veldig viktig	Viktig	Mindre viktig	Irrelevant	Lett gjennomførbart
Usable	X				
Efficient		X			
Reliable		X			
Maintainable		X			
Comprehensible	X				
Testable			X		
Flexible			X		
Interoperable			X		
Correct	X				

Tabell 9: Prioriteringer av arkitektur kriterier

Gruppen har valgt å prioritere kriterier for arkitektur som vist i tabell 9. “Usable”, “comprehensible” og “correct” har blitt markert som “veldig viktig”. Grunnen til dette, er at de er nært knyttet til brukervennlighet. Som nevnt i analysen av applikasjonsdomenet, er brukervennlighet en viktig del av prosjektbeskrivelsen, og avgjørende når det gjelder utviklingen av en webapplikasjon. Måten brukerne oppfatter systemet, er grunnleggende for systemets suksess. Gruppen har derfor satt hovedfokus på at webapplikasjonen er brukbar, enkel og korrekt (Mathiassen et al., 2018, s. 180-181).

Kriteriene: “Efficient”, “reliable” og “maintainable” er vurdert som “viktig”. Et system som er mulig å vedlikeholde, muliggjør stadige endringer og tilpasninger i systemet. Dette kan videre øke effektiviteten til webapplikasjonen. Lav effektivitet kan føre til frustrasjon hos brukere, og dermed resultere i et lite brukervennlig system. Dette er derfor en viktig prioritering i gruppens system. Det er også viktig at systemet er “reliable”, slik at det er mulig å forstå hva som vil skje ved å trykke på en knapp. Disse tre kriteriene blir vurdert som “viktig” ettersom de har en betydelig del i gruppens system (Mathiassen et al., 2018, s. 180).

De resterende kriteriene: “Flexible”, “interoperable” og “testable” blir vurdert til “mindre viktig”. Dette er fordi tidsrammen setter begrensninger for prosjektet, og gruppen måtte dermed nedprioritere disse (Mathiassen et al., 2018, s. 187-188).

4.5.2 Prioriteringer av kriterier for brukergrensesnitt

Kriterier	Veldig viktig	Viktig	Mindre viktig	Irrelevant	Lett gjennomførbart
Familiarity	X				
Consistency			X		
Navigation		X			
Feedback		X			
Recovery		X			
Visibility	X				

Tabell 10: Prioriteringer av brukergrensesnitt kriterier

Gruppen har prioritert kriteriene for brukergrensesnitt som vist i tabell 10. “Familiarity” og “visibility” er prioritert som “veldig viktig”. Ved å gjøre grensesnittet kjent og intuitivt, vil brukere enkelt kunne forstå webapplikasjonen, noe som gjør den tilpasset målgruppen. Hele Norges befolkning skal kunne ta i bruk webapplikasjonen, noe som gjorde dette kriteriet til en viktig prioritet. Gruppen har derfor brukt symboler (figur 6) brukeren vil forstå funksjonen til, ettersom dette blir brukt i andre webapplikasjoner også. “Visibility” er også et sentralt kriterium. Et tydelig grensesnitt med lett tilgjengelig informasjon, vil gjøre det enkelt for brukere å lokalisere seg frem på webapplikasjonen (Mathiassen et al., 2018, s. 117). Gruppen har derfor valgt å lage designet så enkelt som mulig (figur 6), slik at det ikke er vanskelig å finne ønsket funksjon.

Kriteriene: “navigation”, “feedback” og “recovery” er prioritert som “viktig”. Dette er fordi kriteriene bidrar med å skape en brukervennlig opplevelse. God navigasjon gjør det enkelt for brukeren å bevege seg gjennom systemet. Dette er viktig for å øke effektiviteten, og gjøre systemet forståelig. Som vist i figur 6, er det tydelig hvor man må trykke for å oppnå ønsket funksjon. Klar feedback holder brukeren informert om systemets tilstand, slik at de vet at deres handlinger har en effekt. Hver knapp har derfor en tydelig funksjon i gruppens webapplikasjon. Dersom brukeren for eksempel trykker på “meld inn endring”, vil man komme videre til en annen side. Recovery funksjonen sikrer at brukeren kan rette opp i feil, og gå tilbake dersom det er nødvendig (Mathiassen et al., 2018, s. 117-123). Dersom brukeren trykker på “tilbake”, vil sist besøkte side i webapplikasjonen komme frem igjen.

“Consistency” er prioritert som “mindre viktig” i gruppens system. Likevel er kriteriet tatt med i betraktning ved utformingen av webapplikasjonen. Selv om kriteriet har en lavere prioritet enn de andre prinsippene, er det fortsatt nyttig for å opprettholde en sammenhengende brukeropplevelse, med god flyt (Mathiassen et al., 2008, s. 117).

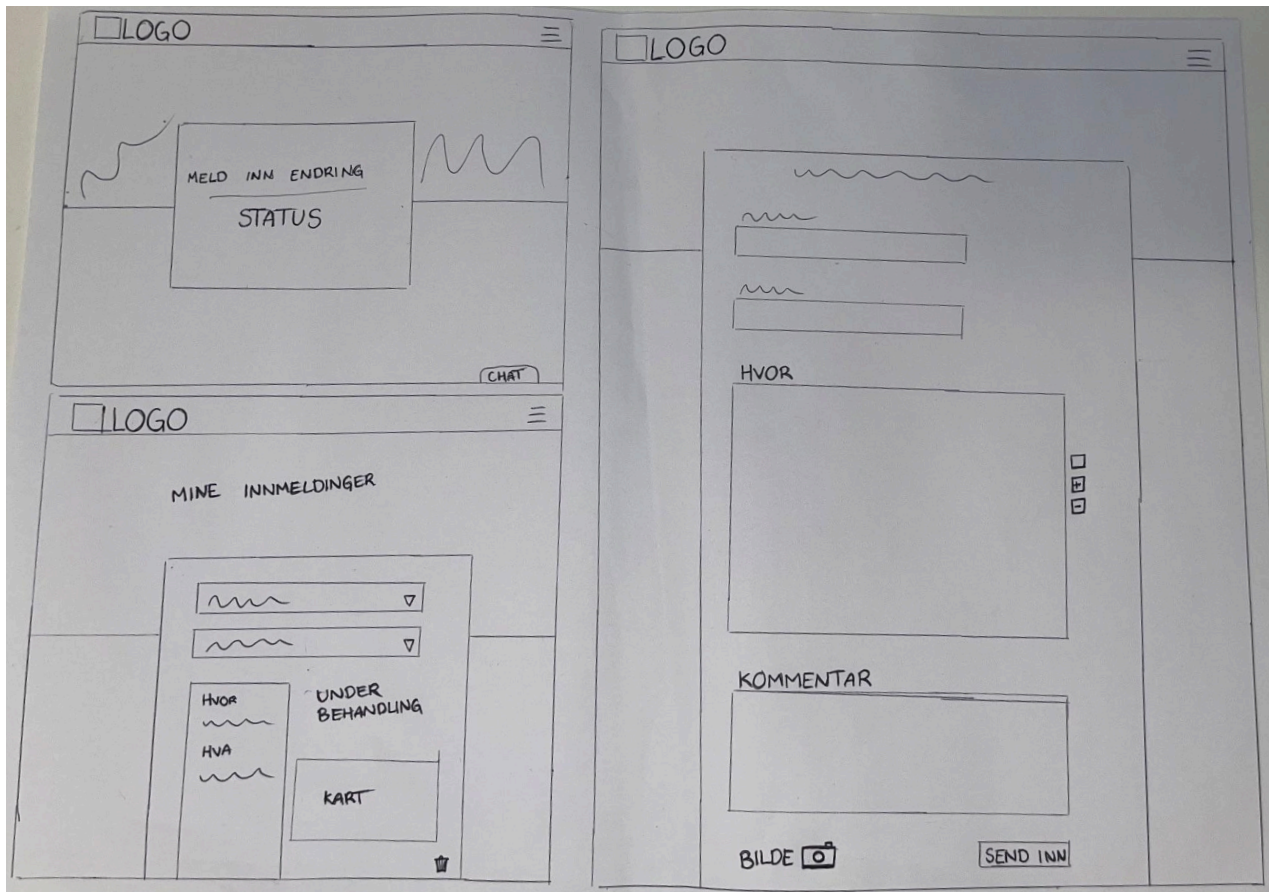


Figur 6: Fra Prototypen

4.6 Sketching

Sketching er en viktig del av designprosessen. Det går ut på å lage simple tegninger, som gjør det enkelt å visualisere ulike ideer. På denne måten blir det å identifisere hvilke ideer som fungerer, og hva som må utelukkes, langt mer effektivt (Stoffel, 2021).

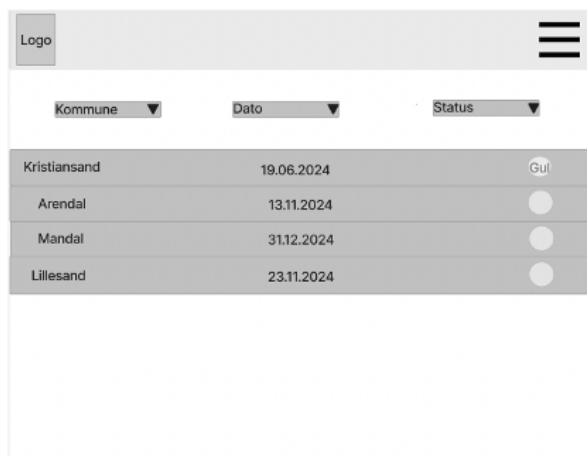
I denne fasen er det viktig å tenke på hvordan man best mulig fremstiller de ønskede funksjonene, og hvordan man kan oppfylle de ulike design kriteriene. For å kunne ta i bruk flere gode ideer, bestemte gruppen seg for å starte med å lage hver sin sketch. Det ble gjort på denne måten for at hvert enkelte gruppemedlem skulle reflektere over mulige løsninger, slik at ulike tanker og synspunkt skulle komme frem. Gruppen diskuterte deretter de ulike ideene, for å videre kunne lage en felles sketch (figur 7). På denne måten fikk gruppen en felles visjon på ønsket design. Sketchen viser hjemmesiden oppe til venstre. Under, er siden for “status”, altså en oversikt over brukerens innmeldinger, og til høyre er siden for “meld inn endringer”.



Figur 7: Fellessketchen

4.7 Wireframe

Wireframe er et element som visualiserer systemets struktur, layout, UI og navigering uten å være for detaljert og spesifikt. Det finnes tre ulike versjoner av wireframe: low-fidelity, mid-fidelity og high-fidelity. De forskjellige versjonene viser ulike stadier i designprosessen, og for hver versjon vil wireframe-modellen bli mer detaljert og utviklet (Figma, u.d).

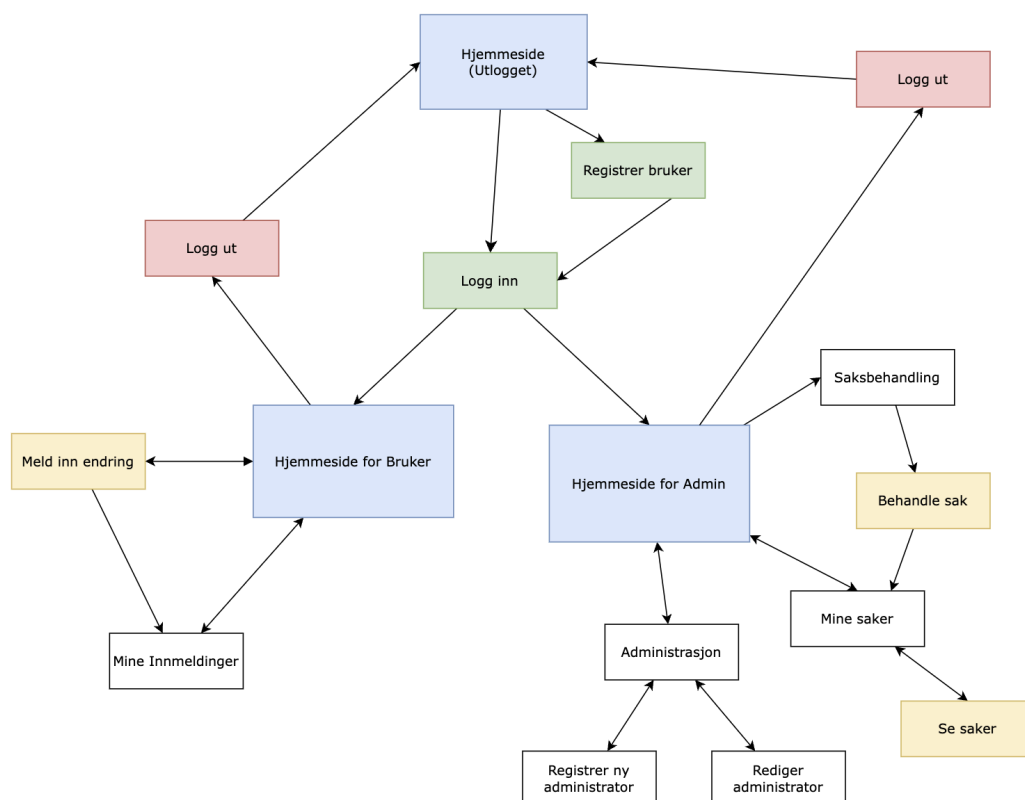


Figur 8: Wireframe for “Alle innmeldinger” siden for en Saksbehandler

Gruppen laget en high fidelity wireframe til systemet. Ved å lage en wireframe fikk gruppen en felles grunnforståelse over hvordan systemet vil se ut, og det ble lettere å oppdage potensielle designmangler tidlig i prosessen. Wireframe var også et godt verktøy å bruke for å fokusere på brukerens behov fremfor design detaljer. Figur 8 viser hvordan siden om alle innmeldinger vil se ut for en saksbehandler. Målet var å gjøre siden både intuitiv og lett å forstå. Bruken av wireframe forenklet denne prosessen fordi den sørget for at selve grunnstrukturen til systemet var intuitivt. Et eksempel på dette er at sorteringsboksene ble plassert godt synlige med et symbol ved siden av. Dette symbolet skal symbolisere at det finnes flere valgalternativer. Wireframe hjalp gruppen til å lage en intuitiv og enkel grunnstruktur som igjen la grunnlaget for en brukervennlig løsning.

4.8 Navigation map

Navigation map demonstrerer hvordan en bruker vil kunne bevege seg gjennom et system. Det inkluderer alle innganger og utganger, altså alle mulige veier brukeren kan bevege seg gjennom systemet. Modellen gir dermed et oversiktlig bilde av hvordan de ulike elementene og sidene henger sammen (Sciencedirect, u.d.). Ettersom systemet skal være mulig å bruke av hele Norges befolkning uavhengig av teknologiske ferdigheter, er det viktig at brukerne kan navigere seg gjennom systemet på en enkel og problemfri måte. God navigasjon resulterer i effektivisert arbeid, og brukerne vil dermed få rask tilgang på informasjon for å løse de nødvendige oppgavene (Benyon, 2019, s. 117).



Figur 9: Navigation map

Navigasjonskartet blir illustrert med bokser og piler som viser alle mulige veier brukeren kan gå. Dette er demonstrert i gruppens modell (figur 9). Webapplikasjonens startposisjon er på hjemmesiden, hvor brukeren ikke er logget inn. For å kunne komme videre i systemet må brukeren logge inn eller registrere en bruker. Grunnen til dette, er fokus på sikkerhet og forebygge at brukere ikke melder inn for mye useriøse henvendelser. Videre vil brukeren komme til hjemmesiden, hvor det er forskjell på om brukeren er innlogget som vanlig bruker eller admin. Dette vil være basert på om innloggingen skjer med en av Kartverkets egne mailer eller ikke. Hvis brukeren ikke er logget inn som admin, vil den kun ha tilgang til å melde inn endringer eller se egne innmeldinger.

Hjemmesiden for admin brukere vil ha litt forskjellige funksjoner, hvor de vil ha en funksjon for å saksbehandle, se egne pågående saksbehandlinger og administrere brukere i systemet. Sidene for administrasjon og mine saker har også en lett synlig tilbakeknapp, noe som er viktig for "recovery" brukergrensesnitt kriteriet. Som vist i figur 10, vil det alltid være mulig å åpne sidemenyen, noe som gjør at brukeren alltid vil ha tilgang til å logge ut.

Gruppen valgte å ta i bruk denne modellen ettersom den bidrar til å skape en felles forståelse over hvordan systemet vil fungere. Navigasjon er en viktig del av designet, og er derfor en av kriteriene for brukergrensesnitt gruppen har prioritert som "viktig". Modellen bidrar med å

skape et system som har god flyt og høy brukervennlighet, og den ble derfor ansett som en nødvendighet.

4.9 Prototype

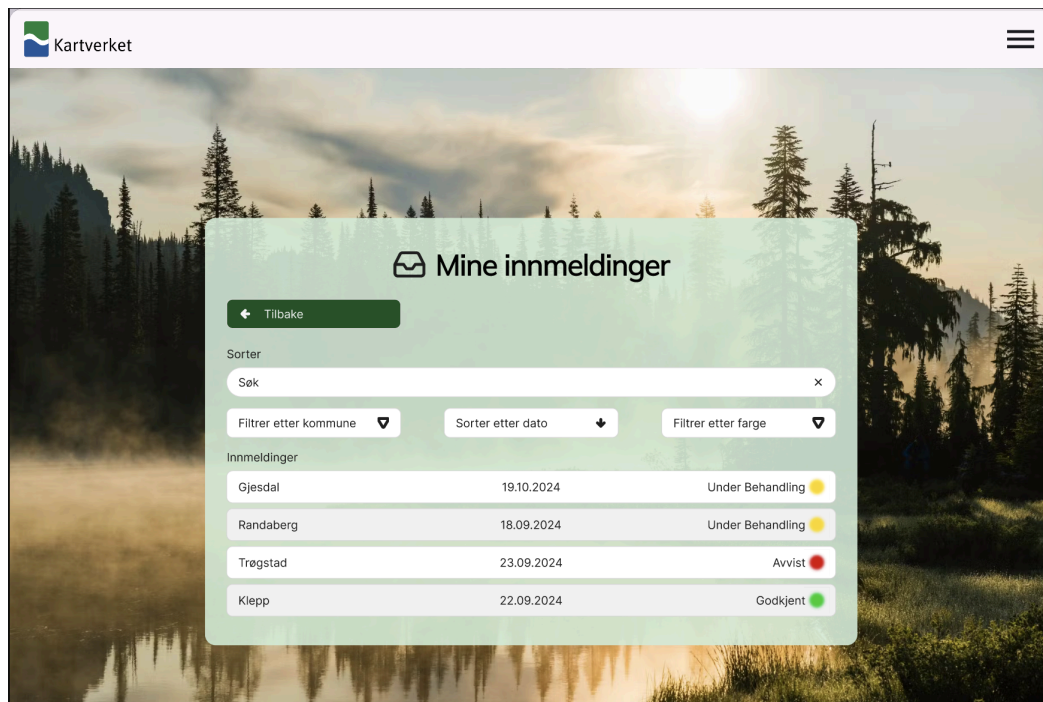
En prototype er en forenklet utgave av designet på webapplikasjonen, som gjør det mulig å teste brukergrensesnitt og design tidlig i utviklingsprosessen. Gruppen sin prototype (se vedlegg 2) inneholder flere klikkbare elementer, bilder, tekst og navigasjon. Modellen demonstrerer dermed både hvordan designet på den ferdigstilte webapplikasjonen kan se ut, og hvordan brukeren kan navigere seg gjennom systemet. Dette er en effektiv og enkel måte å forsikre seg om at alt fungerer som det skal (Coelho, 2021).

For å kunne lage prototypen i Figma, utnyttet gruppen den ferdigstilte wireframen, sammen med navigation map, brukerhistoriene og designkriteriene. Hovedfokuset var å oppfylle "must have" funksjonene fra brukerhistoriene, for å dekke alle kravene til Kartverket. For å skape et brukervennlig system, utnyttet gruppen designkriteriene basert på utvalgte prioriteter. Dette var godt fremstilt i den ferdig utviklede wireframen, som ga en enkel oversikt over mulig design, og alle de ulike sidene gruppen måtte ha med i modellen. For å utforme et mer komplett design, tok også gruppen utgangspunkt i Kartverket sin hjemmeside. Oppsettet og fargene er derfor gjenkjennelig, som oppfyller brukergrensesnitt kriteriet om "familiarity", noe som tydeliggjør at innmeldingssidene henger sammen med resten av nettsiden. Navigasjonskartet demonstrerte hvordan de ulike sidene skulle henge sammen, noe som gjorde videre arbeid med prototypen tydelig organisert og effektivt. Figur 10-12 demonstrerer gruppens prototype.



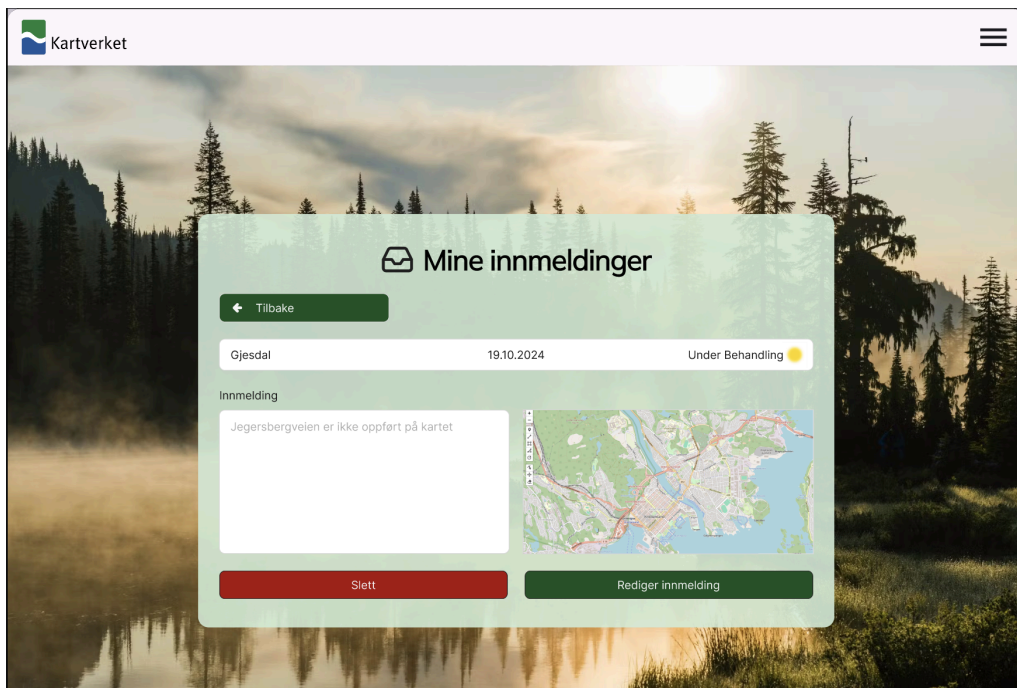
Figur 10: Prototypens hjemmeside

Prototypen demonstrerer et enkelt design, ved bruk av kjente symboler, og konkrete funksjoner, noe som er viktig for både “familiarity” og “visibility” kriteriene. Figur 10 (hjemmesiden) viser en enkel hjemmeside, som inneholder funksjonene: "mine innmeldinger" og "meld inn endring". Ettersom siden kun fremviser disse to mulige funksjonene, er de synlige og lett tilgjengelige. Dette gjør det også lettere for brukeren å navigere seg gjennom systemet, steg for steg. Dersom brukeren velger “mine innmeldinger”, kommer man videre til en ny side (figur 11).



Figur 11: Prototypens side for “mine innmeldinger”

Figur 11 demonstrerer gruppens fokus på "recovery". Dersom brukeren trykker feil, eller ønsker å gå tilbake til forrige side, er det mulig å trykke på "tilbake" oppe i venstre hjørne. “Feedback” er også et kriterie for brukergrensesnitt som har blitt tatt i bruk, for at brukeren skal vite at handlingene deres har en effekt. Derfor har hver knapp en tilhørende funksjon, hvor brukeren får tilbakemelding fra systemet. Dersom brukeren velger "Gjesdal", vil brukeren få en oversikt over den registrerte innmeldingen, som vist i figur 12. Dette oppfyller også kriteriet om “navigation”, som viser at det er lett å navigere gjennom systemet, med tydelige knapper som oppnår brukerens forventning.



Figur 12: Prototypens side for innmeldingen av Gjesdal kommune

Sett sammen, demonstrerer 10-12 et repetitivt design og oppfyller kriteriet om “consistency”.. Hver side likner på hverandre, ettersom blant annet bakgrunn, farger og overskrifter er det samme. Dette skaper god flyt, og dermed et mer brukervennlig system.

5. Scrum

I prosjektarbeid blir det ofte brukt ulike arbeidsmetodikker for planlegging og gjennomføring av prosjekter. To ulike metoder som ofte blir brukt er fossefall-metoden og agil-metoden. Forskjellen på disse metodene er at fossefallmetoden er mer lineær mens agil-metoden er mer iterativ (Świtek, u.d.).

Fossefall-modellen setter søkelys på langsiktige og detaljerte planer, og dette gjør at modellen ofte blir sett på som mer rigid (Holte Academy, u.d.). Grunnen er at de langsiktige planene kan gjøre det vanskeligere å gjøre endringer underveis. Noe som også kan føre til at risikoen for større feil økes. Prosjektprosessen skal foregå i en lang tidslinje, hvor planen for hele prosjektet skal være klart før man har startet. Denne tidslinjen blir delt inn i ulike bolker, og ettersom man blir ferdig med de ulike bolkene vil man komme videre på tidslinjen. Det er dette som skaper fossefall-effekten (Holte Academy, u.d.). Fossefallmetoden kan egne seg godt til prosjekter hvor det ikke forventes endringer underveis, og hvor kravene fra en kunde eller interessent er klare.

Agil-metoden har derimot mer fokus på å dele prosjektet inn i mindre deler med bruk av kortere tidsintervaller, og den legger ikke vekt på langsiktige planer (Holte Academy, u.d.). En type metode som følger agil-filosofien er Scrum. Scrum er et smidig projektrammeverk som blir brukt i prosjekter, og det gjør at et team kan arbeide fleksibelt og iterativt. Metoden deler prosjektprosessen inn i mindre og håndterbare deler, og disse delene blir kalt for sprints (Blisten, 2024). Dette er noe som kan være spesielt gunstig om et prosjekt er komplekst og har krav eller mål som ofte endres underveis.

5.1 Hvorfor velge scrum?

Scrum metoden kommer med en del ulike fordeler, og en av disse er at produktiviteten blir økt. Som nevnt tidligere er sprintene korte perioder med klart definerte og håndterbare oppgaver (Blisten, 2024). Dette gjør at de enkelte teammedlemmene gjerne blir mer målbevisste og holder fokus bedre, noe som igjen kan være med å fremme produktiviteten. Andre fordeler med scrum er at det kan være med å minske stress, skape større eierskapsfølelse og forbedre arbeidsmiljøet. Grunnen til dette er at scrum-metodikken har fokus på selvorganiserende arbeid (Blisten, 2024). Dette kan gjøre at teamene og de enkelte medlemmene får mer eierskap og oversikt over prosjektet og oppgavene sine. Videre kan dette føre til mindre stress og bedre arbeidsmiljø. Scrum fremmer også kommunikasjonen og sørger for høyere kvalitet på leveransene (Blisten, 2024). Kommunikasjonen kan oppleves bedre på grunn av de daglige møtene som blir holdt under de ulike sprintene. Dette er noe som kan være med å øke kvaliteten. En annen ting som øker kvaliteten er at det legges fokus på et tett samarbeid med kunder og interessenter (Blisten, 2024). Dette kan være positivt ettersom det sørger for at deres behov blir ivaretatt.

Metoden har et fast oppsett og består av tre hovedområder: roller, artefakter og hendelser (Rud, 2024). De ulike rollene i scrum er scrum master, product owner og scrum team, og disse rollene har ulike oppgaver og områder. Rollene har også ulike hendelser som de deltar på; sprint, daily scrum, sprint planning, sprint review og sprint retrospective. Disse hendelsene er det som skaper den iterative prosessen som prosjektet går igjennom. Artefaktene er det som er med å styre og strukturere prosjektprosessen, og det består av product backlog og sprint backlog (Rud, 2024). Videre beskrivelse av Scrum oppsettet står i Tabell 11 & 12 (Karlsen, 2021, s. 278-281).

Roller	Beskrivelse	Aktuelle hendelser
Scrum Master	Passer på at scrum blir fulgt, og sørger for at mulige problemer eller uklarheter blir løst.	Alle hendelser

Product Owner	Representerer alle interessenter og kunder av prosjektet. Har også ansvar for å håndtere og oppdatere product backloggen.	Sprint planning og Sprint review
Scrum Team	Utfører oppgavene i product backloggen og jobber selvorganiserende for å oppnå målene i sprintene. Har generelt ansvar for produktutviklingen.	Sprint planning, Daily scrum, Sprint review, Sprint retrospective

Tabell 11: Rollene i SCRUM

Hendelser og artefakter	Beskrivelse
Product Backlog	En prioriteringsliste som inneholder alle kravene i IT-løsningen og er gitt av produkteier(e). Kan hele tiden justeres og vil være i utvikling.
Sprint Backlog	En prioriteringsliste med oppgaver og gjøremål som er basert fra kravene i product backloggen. Oppgavene gjør kravene til funksjonaliteter og dette bør være tidsestimert.
Sprint	Periode med maksimal varighet på fire uker. I denne perioden skal det være daily scrum og sprint backloggen skal holdes oppdatert.
Daily Scrum	Korte møter på ca 15 minutter. En går igjennom viktige spørsmål om status på oppgaven og teammedlemmene må svare på dem.
Sprint Planning	Et møte hvor teamet planlegger arbeidet for den kommende sprinten. En finner ut hvilken produktkrav fra product backloggen som skal være med i sprint backloggen.

Sprint Review	Et møte hvor teamet viser frem hva som har blitt gjennomført og ikke i den gjeldende sprinten. Interessenter og kunder kan også komme med innspill og eventuelle endringsønsker.
Sprint Retrospective	Et møte hvor en ser tilbake og reflekterer over sprinten som har vært. En tenker over hva som har fungert bra og dårlig, og hva som eventuelt kan bli bedre til neste sprint.

Tabell 12: Hendelser og artefakter i SCRUM

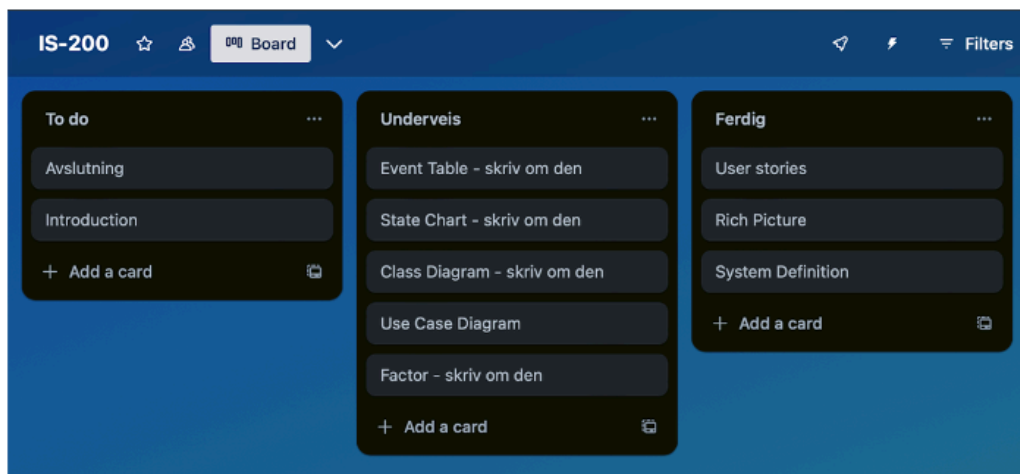
5.2 Pre-Sprint: 26.08.24 - 08.09.24

Gruppen brukte pre-sprinten til å få en forståelse for hva oppgaven innebar og hva som måtte gjøres. En av de første tingene som gruppen måtte gjøre var å sette seg inn i metodikken Scrum. Scrum var noe som var ukjent for de fleste gruppemedlemmene og det var derfor viktig å få en forståelse av hvordan det skulle praktiseres. En av tingene som hjalp gruppen til å få en bedre forståelse var ulike øvelser i form av rollespill. Dette gjorde at gruppemedlemmene fikk prøvd og øvd seg på de ulike rollene.

Videre ble gruppen enige om hvem som skulle være Scrum-master. Scrum-masteren fikk et ansvar for å sette opp en plan for hva som måtte være klart til Sprint 1, og denne planen ble satt inn i programvaren Trello. Trello er et verktøy som kan brukes for å visualisere oppgaver og administrere prosjekter (Trello, u.d.). At gruppen tok i bruk Trello gjorde det lettere å holde en oversikt over hvilke oppgaver som var gjort og ikke.

Pre-sprinten ble hovedsakelig brukt til å planlegge hvordan arbeidet skulle foregå generelt sett. Dette medførte at scrum-metodikken ikke ble benyttet fullt ut og det var heller ikke behov for Sprint Retrospective møte.

5.3 Sprint 1: 09.09.24 - 22.09.24



Figur 13: Kanbanboard for sprint 1

5.3.1 Sprint Planning

For å planlegge Sprint 1 valgte gruppen igjen å bruke Trello. På Trello la gruppen inn alle gjøremålene fra de ulike fagene og dette ble da gruppens Sprintbacklog (Figur 13). Det ble også enighet om at Sprintbackloggen skulle deles opp og sorteres ut ifra hvilket fag gjøremålene tilhørte. Dette gjorde at Sprintbackloggen var systematisk og ryddig, men det førte også til at gruppen ikke fikk en fullstendig oversikt.

En ting som gruppen forsikret seg om var at Sprintbackloggen inneholdt passende mengde arbeidsoppgaver fra Productbackloggen. Likevel opplevde noen av gruppemedlemmene at arbeidsomfanget ble for stort. Dette var på grunn av at gruppen glemte å sette både tidsestimater og fordele gjøremålene. Å sette tidsestimater handler om å estimere hvor lang tid ulike oppgaver eller prosjekter tar, og det er en viktig del av prosjektprosessen (Matyska, 2024).

Trello bidro likevel positivt ved å tydelig vise hvilke oppgaver som var fullført og ikke. Dette gjorde at gruppen fikk bedre innsikt i fremdriften. Etter disse erfaringene forstod gruppen hvor viktig det er å tydeliggjøre arbeidsfordelingen og tidfeste oppgaver for å sikre en bedre strukturert arbeidsprosess i de fremtidige sprintene.

5.3.2 Daily Scrum og arbeidet

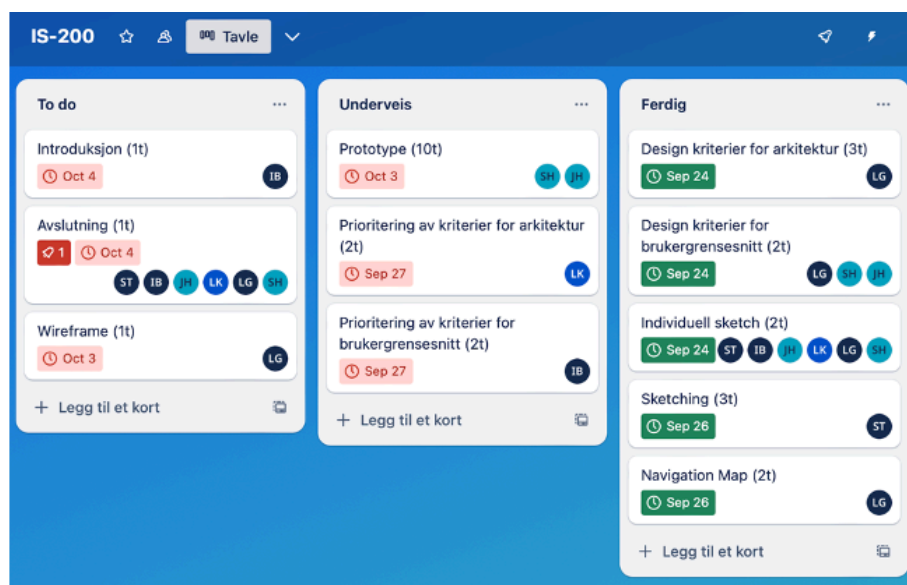
Under sprint 1 gjennomførte gruppen Daily Scrum og q&a. I Daily Scrum fikk gruppen reflektert over status på arbeidet så langt og lagt en visjon for arbeidet fremover. Gruppen

hadde Daily Scrum ca 2-3 ganger i uken. Dette var noe som fungerte bra, fordi da fikk de individuelle gruppemedlemmene en mer helhetlig oversikt over arbeidet. Det sørget også for at alle hadde kontinuerlig med arbeid mellom møtene. Q&A ble utført med kunden og de andre gruppene, noe som ga en bedre innsikt i hva kunden faktisk ønsket. Spørsmålene fra de andre gruppene bidro også til en bredere forståelse av oppgaven. Gruppen opplevde det som svært nyttig å ha en q&a med kunden i sprint 1. Det ga muligheten for at endringer kunne skje tidlig i prosjektet, noe som var med på å fremme den iterative tilnærmingen som er kjernen i Scrum (Eby, 2019).

5.3.3 Sprint Review og Sprint Retrospective

Sprint Review gjorde at gruppen fikk en bekreftelse på at de var på rett vei. Dette gjorde at gruppen ble mer sikre på hvordan de hadde lagt opp og planlagt sprinten. Gruppen fikk tilbakemelding på mangel av tidsestimering. Dette var allerede kjent for gruppen fordi det hadde skapt en del komplikasjoner under sprinten. En av komplikasjonene var som nevnt tidligere at det ble mer arbeid på enkelte av gruppemedlemmene. Det gjorde også at gruppen fikk mindre kontroll over fremdriften av prosjektet, som igjen kunne ført til at ulike forsinkelser eller mangler hadde blitt oppdaget for sent. En annen konsekvens var at det var vanskelig å prioritere oppgavene riktig. Mangel på tidsestimat på oppgaver gjorde at oppgaver som var mer tidskrevende ble nedprioritert og motsatt. Videre skapte dette en ujevn arbeids flyt og unødvendig stress.

5.4 Sprint 2: 23.09.24 - 06.10.24



Figur 14: Kanbanboard for sprint 2

5.4.1 Sprint planning

Basert på erfaringene fra sprint 1, valgte gruppen igjen å bruke Trello for å planlegge sprint 2. Denne gangen besto sprintbackloggen av mindre deloppgaver, som vist i Trello boardet for is-200 (Figur 14). Dette gjorde at gjøremålene ble mer spesifisert og enklere å fordele, og i tillegg økte det effektiviteten til gruppa.

I sprint 1 erfarte gruppen at mangelen på tidsestimering bidro til ujevn arbeidsbelastning og feil prioriteringer. Som et resultat av dette valgte gruppen å sette mer søkelys på tidsestimering. Hver oppgave ble tidsestimert og satt inn i Trello. Dette gjorde at gruppen fikk en bedre forståelse av tidsforbruk og det var med på å sikre en mer realistisk planlegging.

5.4.2 Daily Scrum og arbeid

Under sprint 2, tok gruppen videre det som ga gode resultater i sprint 1 og en av disse tingene var Daily Scrum. Basert på tidligere erfaringer ble Daily Scrum gjennomført 2-3 ganger i uken, og dette ga alle gruppemedlemmene en helhetlig og oppdatert oversikt over statusen til sprinten. I tillegg til dette gjorde det også lettere for gruppen å gjøre eventuelle justeringer underveis.

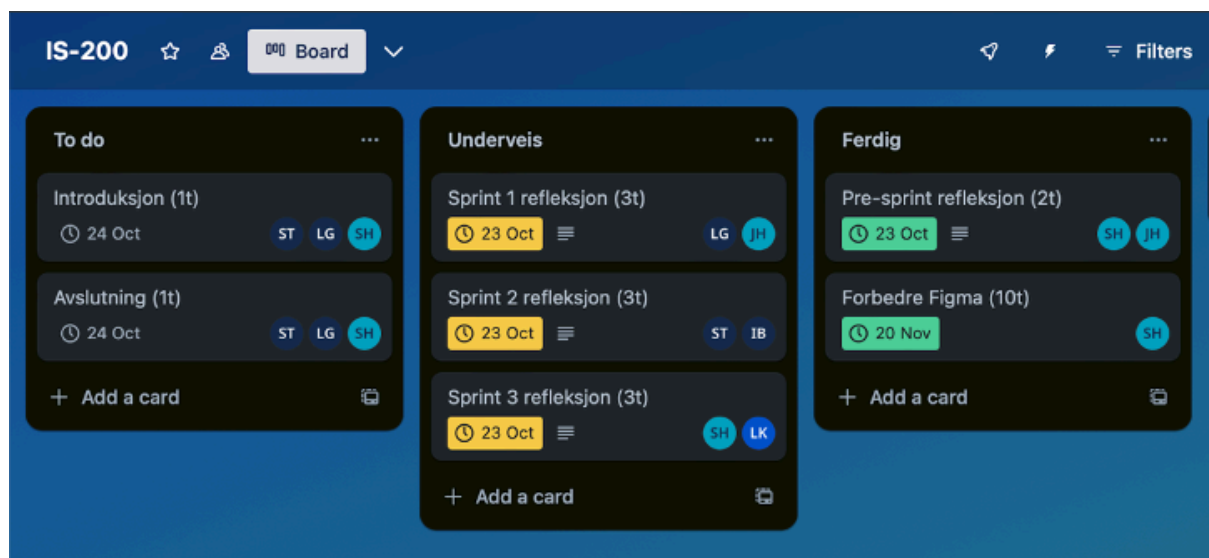
Det ble også gjennomført q&a og denne gangen hadde gruppen forberedt ulike spørsmål som var rettet mot det påbegynte arbeidet. Dette var viktig fordi at gruppen allerede hadde startet med å utvikle ulike sketcher for ønsket design. Ved å stille spørsmål om krav og nødvendigheter kunne gruppen sikre at arbeidet var fokusert på de rette prioriteringene og kravene. Noe som kunne effektivisere arbeidsprosessen fordi man unngikk å bruke unødvendig tid på irrelevante funksjoner.

5.4.3 Sprint Review og Sprint Retrospective

I Sprint Review 2 fikk gruppen igjen tilbakemelding på at det var lurt å prioritere tidsestimering. Dette var noe som gruppen hadde forsøkt å fokusere mer på, men som viste seg å være mer utfordrende enn forventet. Det som var utfordrende, var å sette rett estimering på de ulike oppgavene og gjøremålene. Noen av gjøremålene ble feilberegnet og et eksempel på dette er at gruppen ikke fikk tid til å lage en high fidelity Wireframe. Gruppen hadde nemlig beregnet alt for lite tid til denne oppgaven og dermed ble den nedprioritert. Nedprioriteringen medførte at gruppen bare hadde tid til å lage low fidelity wireframe. Denne erfaringen ga gruppen en forståelse for hvor viktig det var å tidsestimere presist. Videre kom gruppen frem til å prioritere mer tid på planlegging av estimeringen.

Noe som derimot hadde fungert bra i sprint 2 var kommunikasjonen i gruppen. Kommunikasjonen ble styrket gjennom Daily Scrum og var en avgjørende faktor for å opprettholde fremdrift selv i de mer stressende periodene.

5.5 Sprint 3: 07.10.24 - 20.10.24



Figur 15: Kanbanboard for sprint 3

5.5.1 Sprint Planning

Planleggingen av Sprint 3 ble gjennomført på en liknende måte som de forrige sprintene, det eneste var at den hadde enkle justeringer basert på ny erfaring og tilbakemelding fra Sprint Review. Den største endringen i planleggingen av Sprinten var at gruppen prioriterte å bruke enda mer tid på tidsestimering. På denne måten kunne tidsestimeringen bli enda mer presis, slik at risikoen for feilestimeringer minsket. Som tidligere la gruppen inn tidsestimater for de ulike oppgavene i Trello (Figur 15). Det ble også planlagt at Scum Master fikk hovedansvaret for rapporten i is-200. Grunnen til dette var at gruppen innså hvor viktig det var med en klar lederrolle i dokumenteringen og oppfølging av Scrum.

5.5.2 Daily Scrum og arbeidet

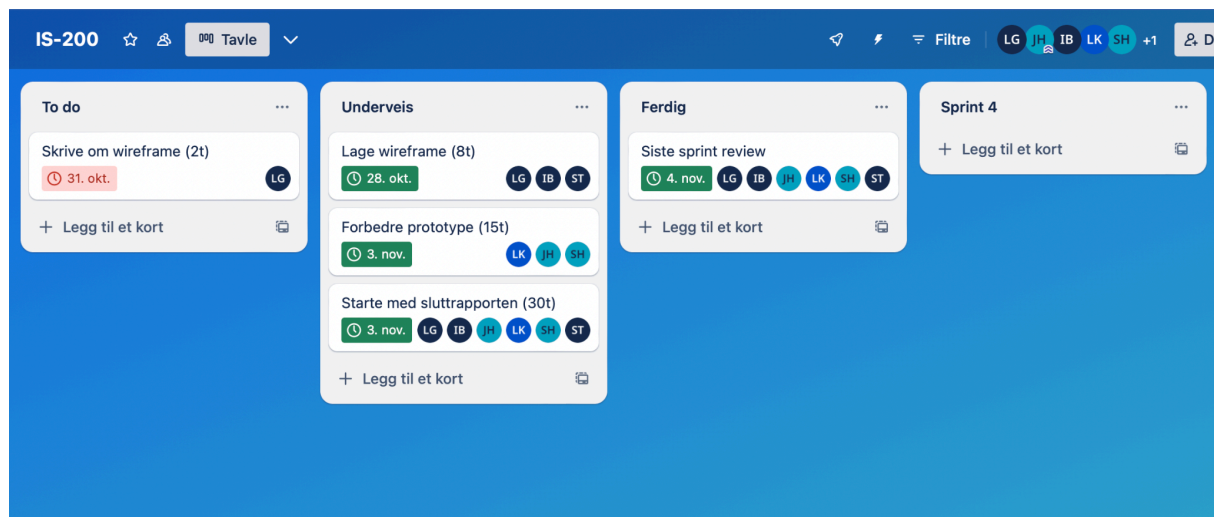
Gruppen fortsatte med å holde Daily Scrum 2-3 ganger i uken, og som tidligere ga dette gode resultater. Blant annet gjorde dette at hvert gruppemedlem fikk en god og helhetlig oversikt

over alle fagene. Gruppens fokus på å tidsestimere mer presist førte til at arbeidsmengden holdt seg jevn under hele sprinten.

5.5.3 Sprint review og Sprint Retrospective

Etter Sprint View oppdaget gruppen at det burde lages en high fidelity wireframe. Dette var fordi systemet fikk behov for flere sider enn opprinnelig planlagt, og disse sidene trengte en wireframe for videre utvikling. Gruppen fant da ut at det kunne være gunstig å lage en high fidelity wireframe til hele systemet. På denne måten kunne man få frem hele designprosessen. I Retrospective møte ble det bestemt at Daily Scrum burde være 3-4 ganger i uken istedenfor 2-3 ganger. Arbeidsomfanget hadde økt og det var forventet at det kom til å øke mer. Dette skapte et behov for å møtes oftere, slik at gruppemedlemmene kunne holde seg oppdatert, men også stille eventuelle spørsmål.

5.6 Sprint 4: 21.10.24 - 03.11.24



Figur 16: Kanbanboard for sprint 4

5.6.1 Sprint planning

Ved planleggingen av Sprint 4 hadde gruppen opparbeidet seg en del erfaring om passende arbeidsmengde, rettferdig arbeidsfordeling og rett tidsestimering. Gruppen hadde også blitt bedre på å sette opp et ryddig og systematisk Trello board (Figur 16). Dette gjorde at alle gruppemedlemmene fikk en klar oversikt over prosjektprosessen og dermed ble selve arbeidet forbedret. Videre ble det bestemt innad i gruppen at gruppen skulle følge samme arbeidsstruktur som i den tidligere sprinten.

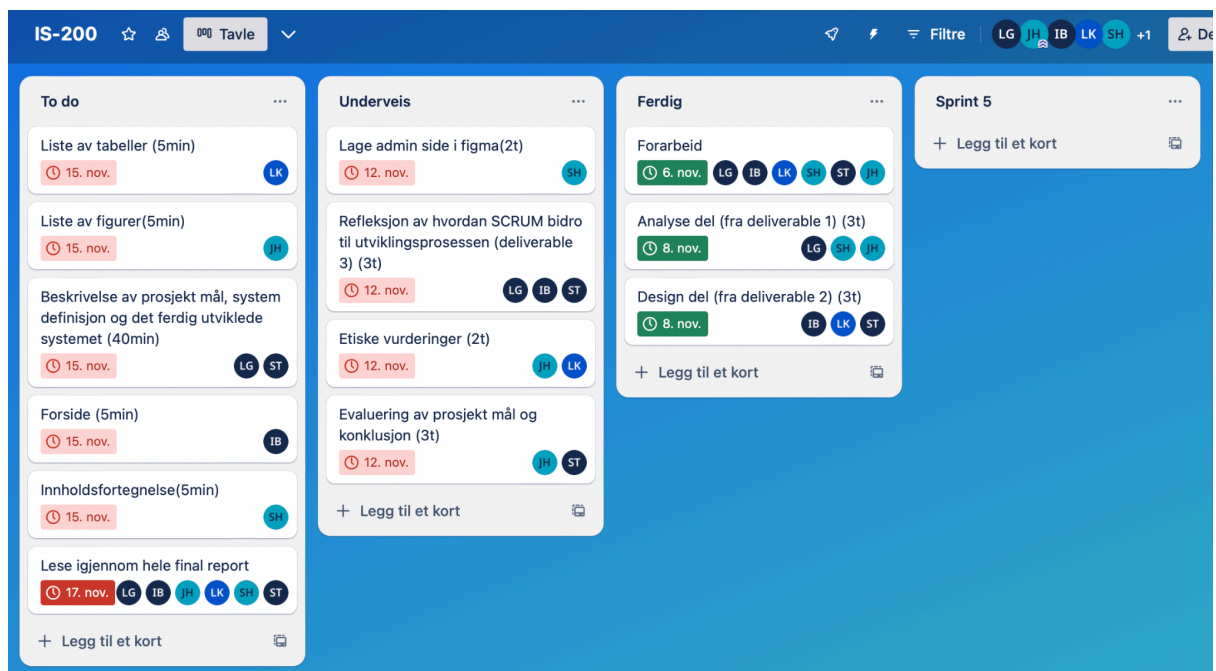
5.6.2 Daily Scrum og arbeid

I Sprint 3 ble gruppen enige om at Daily Scrum møtene skulle være 3-4 ganger i uken istedenfor 2-3 ganger. Som nevnt tidligere var dette fordi at arbeidsomfanget hadde økt og det var større behov for oppfølging. Oppfølgingen gjorde at gruppen fikk god oversikt over statusen på de ulike gjøremålene i forhold til deres tidsestimat. Det ble også satt mer fokus på sluttrapporten for å unngå at alt skulle hope seg opp rett før innleveringsfrist. Dette var også med på å sikre kvaliteten på arbeidet.

5.6.3 Sprint Review og Sprint Retrospective

Gruppen opplevde det som svært nyttig å øke antall Daily Scrum møter. Hvert gruppemedlem fikk holde seg oppdatert på selve prosjektprosessen og fikk en bedre oversikt over alle fagene. Det bidro også til en dypere forståelse av hvordan alle de ulike fagene hang sammen og i tillegg forbedret det samarbeidet. Grunnet de hyppigere møtene gjorde dette at samarbeidet i gruppen føltes tettere.

5.7 Sprint 5: 04.11.24 - 17.11.24



Figur 17: Kanbanboard for sprint 5

5.7.1 Sprint planning

De gode opplevelsene gruppen fikk av sprint 4 resulterte i at gruppen strukturerte sprint 5 på samme måte (Figur 17). Noe av det som var viktig å fokusere på i Sprint 5 var å sette endelige

datoer på de ulike arbeidsoppgavene. Dette var viktig for å sikre at gruppen ble ferdig med oppgavene før endelige innleveringer.

5.7.2 Daily Scrum og arbeid

Gruppen valgte å fortsette med å ha Daily Scrum 3-4 ganger i uken grunnet de gode erfaringene fra tidligere sprint. Under sprinten møttes gruppen flere ganger for å grundig gå gjennom arbeidet som hadde blitt gjort individuelt. Dette ble gjort for å sørge for at alle hadde en oversikt og forståelse for alle oppgavene som hadde blitt gjort.

5.7.3 Sprint Review og Sprint Retrospective

Det var ingen Sprint Review denne sprinten, men det ble likevel holdt et Sprint Retrospective møte. Gruppen reflekterte over hvordan Scrum hadde påvirket gruppearbeidet positivt, men også hva som kunne vært bedre. Scrum sørget for at gruppen hadde kontinuerlig kommunikasjon og at alt av arbeid ble oppfølgt og forstått av alle.

5.7.4 Refleksjon av Scrum

Gjennom bruken av Scrum har gruppen fått gode erfaringer. Metoden bidro til at gruppearbeidet ble mer strukturert og ryddig, samtidig som det forbedret kommunikasjonen i gruppen. Dette var viktige faktorer for at samarbeidet i gruppen skulle foregå på best mulig måte. Strukturen ble mer ryddig og kommunikasjonen ble bedre grunnet de faste hendelsene: Sprint Planning, Daily Scrum, Sprint Review og Sprint Retrospective. Artefaktene product- og sprintbacklog i kombinasjon med Trello, var også med på å strukturere arbeidet. Alt dette gjorde at det ble lettere å jobbe sammen med et prosjekt som en gruppe.

Selv om gruppen erfarte mange gode opplevelser, var det også noen dårlige erfaringer og ting som kunne blitt bedre. En av tingene som var utfordrende var at det var vanskeligere å praktisere Scrum enn først tenkt. Gruppen fikk blant annet ikke praktisere de ulike rollene fullt ut. Rollene ble litt mer vagere enn ønsket, noe som påvirket at man ikke fikk brukt Scrum akkurat slik som det er satt opp til å brukes. En annen ting som gruppen syntes var utfordrende var tidsestimering og rett bruk av Trello. Det som erfartes av dette var at en burde prioritere tidsestimering og dokumentering i Trello høyere, slik at en kunne unngå ujevn arbeidsflyt og misforståelser. Disse erfaringene har gjort at alle gruppemedlemmene har fått mer kunnskap om Scrum og hvordan det praktiseres. Dette er noe som kan være nyttig om man skal benytte seg av Scrum i fremtidige studier eller i arbeidslivet.

5.8 Etikk og universell utforming

Når en skal utvikle et system gjør man mange ulike valg, og noen av disse valgene er relatert til hvordan man kan oppnå et etisk design. Etisk design handler om at et design skal være tilgjengelig og menneskesentrert, respektere datavern, men også motstå manipulerende mønstre (Gosset, 2021). Noe av det som kan hjelpe med å oppnå et etisk design er å prioritere universell utforming.

Universell utforming handler om å planlegge og utvikle tjenester, programmer og produkter på slik måte at flest mulig mennesker kan bruke det på en likeverdig måte (Lid, 2024). Noen grunner til hvordan universell utforming kan hjelpe med å skape et etisk design er at det fremmer inkludering, reduserer diskriminering og ivaretar brukerens verdighet. Dette kan gi flere fordeler, og et eksempel er at man kan få flere brukere av systemet grunnet den økte inkluderingen. Et annet eksempel er at et universelt utformet system gjerne oppleves mer brukervennlig, og dette kan resultere i at flere velger å ta i bruk systemet flere ganger.

Det er flere ulike ting man kan gjøre for å gjøre et design universelt utformet, og noen eksempler er å sørge for god lesbarhet, ha rett fargebruk og lett forståelig navigasjon og layout. I tillegg til dette finnes det noen retningslinjer som skal sørge for at nettsider blir mer tilgjengelige. Disse retningslinjene kalles for WCAG og står for Web Content Accessibility Guidelines (Grolid, 2023).

Designet til gruppen er universelt utformet og brukervennlig på flere måter. Noen eksempler på dette er at det er intuitivt, enkelt å bruke og har et universelt design. I designet har det blant annet blitt tatt i bruk ikoner som skal symbolisere funksjonene slik at navigering blir lettere. Det har også blitt valgt en lesbar typografi og behagelige farger som skaper en kontrast slik at de ikke forsvinner inn i hverandre (Figur 6). I tillegg til dette består navigeringen i systemet av minst mulig tastetrykk og det følger et naturlig mønster som kan være med å gjøre bruken lettere.

Selv om designet inneholder mye av det som fører til en universell utforming, er det noen ting som kunne vært bedre. Det har blant annet ikke blitt implementert noen funksjoner for å ha skalerbar tekst eller gjøre tekst om til tale. Man har heller ikke muligheten til å navigere seg gjennom systemet med tastatur. Dette er noen av tingene som systemet burde hatt for å oppnå en optimal universell utforming, men som ikke har blitt utført grunnet uventet arbeidsmengde og tidsfrister.

6. Evaluering

Gjennom denne rapporten har gruppen tatt stilling til ulike utfordringer, målene og kravene gitt av Kartverket og Universitetet i Agder.

Generelt sett har utførelsen av prosjektet gått bra. Gruppen er enig om at samarbeidet har gått greit, som følge av god oppfølging og kommunikasjon. For hver oppgave underveis, utførte gruppen kritiske vurderinger, i form av diskusjoner. På denne måten fikk gruppen en felles enighet og forståelse av hvordan løsningen skulle bli. Hvert gruppemedlem har gjennomført tildelte oppgaver, noe som har ført til at alle har bidratt i like stor grad. Likevel opplevde gruppen enkelte utfordringer underveis i arbeidet. I perioder ble det litt for mye arbeid, på grunn av dårlig tidsestimering. Dette førte til utfordringer med å få fullført enkelte oppgaver i tide. Dette var heldigvis kun et problem i starten av prosjektet, og tidsestimeringen ble mer vellykket for hver Sprint. Helhetlig, er gruppen fornøyd med sluttproduktet, og mener det oppfyller kravene som er satt fra Kartverket og Universitetet i Agder.

6.1 Analyse

I starten av prosjektet opplevde gruppen utfordringer med å ordentlig komme i gang. En av årsakene var at prosjektet er tverrfaglig og det krever en del informasjon fra ulike emner. En annen grunn var at mye av fagstoffet var nytt, og det var viktig at hvert medlem opparbeidet seg en grunnleggende forståelse, før det ble arbeidet videre i fellesskap. Etter hvert som alle medlemmene hadde fått en god forståelse, dukket det imidlertid opp en felles utfordring, forskjellen mellom problemdomenet og applikasjonsdomenet. De to domenene fremstod som svært like i teorien, men har viktige detaljer som skiller.

For å løse denne utfordringen tok gruppen i bruk forskjellige metoder. Det var alt fra brukerhistorier, FACTOR, klassediagram til MoSCoW-metoden, navigasjonskart og Use Case diagram. FACTOR-metoden var særlig nyttig ved at den tydeliggjorde aspektene som definerte applikasjonsdomenet, mens brukerhistorie hjalp med problemdomenet. Ved å utforme ulike scenarioer ble det lettere for gruppen å få en god forståelse av hva problemdomenet betyr for gruppens prosjekt.

Underveis i utviklingen ble det satt opp frivillige Q&A med produkteier, Kartverket. Gruppen hadde på forhånd forberedt ulike spørsmål knyttet til ulike temaer i prosjektet. Disse Q&A sesjonene var felles for alle grupper og var veldig lærerikt for videre utvikling. Andre gruppers spørsmål tok for seg temaer som gruppen ikke hadde tenkt over, men det kom som en fordel. Noen av svarene var nyttige ettersom de gav god innsikt. Sammen med metodene

og Q&A sesjonene ble det enklere å skape en klar og presis systemdefinisjon. I tillegg ble prosjektmålene mer definerte.

6.2 Design

Design delen av prosjektet var forholdsvis grei å gjennomføre, ettersom gruppen hadde erfaring fra det tidligere faget IS-104, Digital interaksjonsdesign. Dette faget dekket mye av det relevante stoffet, inkludert Benyons design prinsipper, PACT analyse, og verktøy som navigasjonskart, wireframes, og utvikling av prototyper. Med forståelse av disse prinsippene og verktøyene, var det lett for gruppen å diskutere og lage sketcher for designet, tidlig i utviklingsprosessen. Etter flere versjoner og utkast av skisser, gikk gruppen videre til å utarbeide navigasjonskart og wireframes, før arbeidet med prototypen ble påbegynt. Arbeidet med wireframes la et godt grunnlag for en god overgang til utvikling av prototypen.

PACT-analysen fungerte som en veiledende ramme for prosjektet. Gjennom diskusjoner ble de mest relevante prinsippene, brukere, kontekster, og teknologier identifisert. Det ble dermed mer klarhet på hvem webapplikasjonen skulle brukes av, og hva den skulle brukes til. Ut fra analysen, kunne gruppen velge ut de mest sentrale kriteriene for brukergrensesnitt.

Under utvikling av webapplikasjonen, hadde gruppen sterkt fokus på prinsippene om “familiarity” og “visibility” fra Benyons designprinsipper. Gruppen ønsket å sikre at brukerne enkelt kunne navigere og forstå applikasjonen uten behov for lang tilvenning. Samtidig ble synlighet prioritert ved å tydelig fremheve viktige funksjoner, slik at brukeren raskt kunne finne fram til ønsket funksjon, som bidrar til en brukervennlig opplevelse.

6.3 Scrum

Dette gruppeprosjektet har gitt alle medlemmene i gruppen en bedre forståelse av hvordan man kan samarbeide på best mulig måte. Noe av det som har påvirket gruppearbeidet positivt er metodikken Scrum. Gjennom bruken av de ulike hendelsene, artefaktene og rollene har gruppen fått en smidig prosjektprosess. Gruppen har også erfart at god kommunikasjon er en av de viktigste faktorene for å få et godt gruppearbeid. Noe av det som har vært med å fremme kommunikasjonen er Daily Scrum, Sprint Review og Retro Spective. Videre, har gruppen erfart at planlegging og tidsestimering også er viktig i et gruppearbeid og i en prosjektprosess. Dette er noe som gruppen har fått til ved bruk av Sprint Planning og Sprint Retrospective.

Selv om gruppen ikke hadde noe særlig kunnskap om Scrum før dette prosjektet, er gruppen fornøyd med egen innsats. Læringskurven om hvordan man praktiserer Scrum har vært stor. Det har vært gradvise forbedringer fra Sprint til Sprint, og dette er mye grunnet Sprint Retrospective møtene. Retro Spective møtene har fått gruppen til å reflektere over hva som har fungert i Sprinten og hva som kan bli bedre til neste. Dette er noe som er viktig for å sørge for en konstant utvikling og forbedring.

Gruppen har blitt ferdig med flere av gjøremålene (brukerhistoriene) i Sprint Backloggen, men det er enda noen som mangler. Brukerhistoriene som har blitt gjort er de som er blitt prioritert som «must have» i MoSCow-metoden, og de som gjenstår er de brukerhistoriene som ble nedprioritert fra start.

7. Referanse

Atlassian Trello. (u.d.). *Trello gjør det enklere for team å administrere prosjekter og oppgaver*.

Retrieved Oktober 26, 2024, from

<https://trello.com/nb/tour>

Blisten, H. (2024, Oktober 03). *Scrummetodikk - slik funker det*. Retrieved November 10, 2024, from

<https://www.folq.no/blog/scrummetodikk-slik-funker-det/#scrummetodikk---slik-funker-det>

Benyon, D. (2019). *Designing User Experience. A guide to HCI, UX and interaction design (4th ed.)*. Pearson

Coelho, J. T. B. (2021, Mai 28). *Hva er en prototype?* NDLA: Forsiden. Retrieved November 2, 2024,

from <http://ndla.no>

Accessed 10 November 2024.

Dingsøy, T. (2024, Oktober 02). *scrum – utviklingsmetode – Store norske leksikon*. Store norske

leksikon. Retrieved Oktober 28, 2024, from https://snl.no/scrum_-_utviklingsmetode

Dropbox. (u.d.). *Hva er agile metoder innen prosjektstyring?* - Dropbox. Experience Dropbox.

Retrieved November 5, 2024, from

<https://experience.dropbox.com/nb-no/resources/agile-methodology>

Eby, K. (2019, Januar 2). *All about the Iterative Design Process*. Smartsheet. Retrieved Oktober 31, 2024, from <https://www.smartsheet.com/iterative-process-guide>

Figma. (u.d). *What is Wireframing? The Complete Guide [Free Checklist]*. Figma. Retrieved November 1, 2024, from <https://www.figma.com/resource-library/what-is-wireframing/>

Gosset, S. (2021, Oktober 6). *What Exactly Is Ethical Design?* BuiltIn. Retrieved November 4, 2024, from <https://builtin.com/articles/ethical-design>

Grolid, S. Å. (2023, Mai 23). *WCAG – retningslinjer for tilgjengelig netttinnhold - Utvikling*. NDLA. Retrieved Oktober 29, 2024, from <https://ndla.no/nb/subject:5e53694a-c8eb-4871-8558-71523941c28e/topic:0d260948-d2c6-4b8d-9c01-97aac6610961/resource:5f194d9c-49e3-415a-8b6a-a4127003d294>

Hanna, T. (2023, Mai). *What is a State Diagram? Definition from WhatIs.com*. TechTarget. Retrieved November 11, 2024, from <https://www.techtarget.com/searchapparchitecture/definition/state-diagram-state-machine-diagram-or-statechart-diagram>

Holde Academy. (u.d). *Agil eller fossefall – hvilken metode skal vi velge?* Holte Academy. Retrieved November 8, 2024, from <https://www.holteacademy.no/agil-eller-fossefall-hvilken-metode-skal-vi-velge/>

Institute of data. (2023, November 15). *What is Problem Domain and Solution in Software Engineering?* Institute of Data. Retrieved November 9, 2024, from <https://www.institutedata.com/blog/what-is-problem-domain-and-solution/>

Jain, S. (2024, October 11). *MVC Design Pattern*. GeeksforGeeks. Retrieved November 7, 2024, from <https://www.geeksforgeeks.org/mvc-design-pattern/>

Jain, S. (2024, October 18). *Class Diagram | Unified Modeling Language (UML)*. GeeksforGeeks. Retrieved November 11, 2024, from <https://www.geeksforgeeks.org/unified-modeling-language-uml-class-diagrams/>

Kartverket. (2024, August 21). *Kva Kartverket gjer*. Kartverket. Retrieved November 2, 2024, from <https://www.kartverket.no/om-kartverket/kva-kartverket-gjer>

Kaye, S. (u.d.). *Rich Pictures*. Better evaluation. Retrieved Oktober. 28., 2024, from <https://www.betterevaluation.org/methods-approaches/methods/rich-pictures>

Lid, I. M. (2024, September 5). *universell utforming – Store norske leksikon*. Store norske leksikon. Retrieved November 5, 2024, from https://snl.no/universell_utforming

Lucidchart. (u.d.). *UML Use Case Diagram Tutorial*. Lucidchart. Retrieved November 2, 2024, from <https://www.lucidchart.com/pages/uml-use-case-diagram>

Mathiassen, L. (2018). *Object-oriented Analysis & Design* (2.edition ed.). Metodica.

Mätt, T. H. (2023, Februar. 15.). *Brukergrensesnitt*. Retrieved November. 10., 2024, from <https://snl.no/brukergrensesnitt>

Matyska, K. (2022, November 15). *What is the MoSCoW Prioritization method?* Timeular. Retrieved November 4, 2024, from <https://timeular.com/no/blog/moscow-prioritization-method/>

Matyska, K. (2024, July 23). *Time Estimation: Simple Guide to Accurate Project Estimation*. Timeular. Retrieved Oktober 28, 2024, from <https://timeular.com/blog/time-estimation/>

Mikalsen, M. (2024, March 22). *brukerhistorie – Store norske leksikon*. Store norske leksikon. Retrieved Oktober 27, 2024, from <https://snl.no/brukerhistorie>

Parker, A. (2024, Oktober 26.). *UML klassediagramopplæring: Abstrakt klasse med eksempler*. Retrieved November 10., 2024, from <https://www.guru99.com/no/uml-class-diagram.html>

Rehkopf, M. (u.d.). *What is a kanban board*. Retrieved Oktober 27., 2024, from <https://www.atlassian.com/agile/kanban/boards>

Rud, N. (2024, April 10.). *Understanding the Scrum Framework: Roles, Events and Artifacts*. Retrieved November 18., 2024, from <https://attractgroup.com/blog/scrum-framework-scrum-roles-artifacts-and-events/>

Science Direct. (u.d.). *Navigation Map*. Retrieved November 14., 2024, from <https://www.sciencedirect.com/topics/computer-science/navigation-map>

Sheldon, R. (2023, September). *Model-view-controller (MVC)*. Techtarget. Retrieved November 12., 2024, from <https://www.techtarget.com/whatis/definition/model-view-controller-MVC>

Stoffel, C. (2021., Februar. 22.). *The role of sketching in the design process*. Retrieved Oktober 28., 2024, from <https://toddassoc.com/the-role-of-sketching-in-the-design-process/>

Switek, M. (u.d.). *Agile vs. Waterfall: en oversiktlig guide til å velge riktig metodikk*. Retrieved November. 10., 2024, from <https://teamdeck.io/nb/ressurser/smidig-vs-fossefall-en-oversiktlig-guide-til-a-velge-riktig-metodikk/>

Team Asana. (2024, Januar. 9.). *What is a sprint backlog? How to create one, with examples.* asana.

Retrieved Oktober. 26., 2024, from <https://asana.com/resources/sprint-backlog>

Wass, S. (26. August 2024). *Lecture 3: Analysis I.* [Systemanalyse og systemutvikling].

8. Vedlegg

8.1 Vedlegg 1:

Prosjektoppgave UiA - Kartverket

<https://uia.instructure.com/courses/16039/assignments/66070>

8.2 Vedlegg 2:

Link til Prototypen:

<https://www.figma.com/proto/4lyxq5is1oa0rHxucV1YQy/Kartverket?page-id=0%3A1&node-id=230-1075&node-type=canvas&viewport=3457%2C444%2C0.34&t=e4JNJNgpqEywLFOI-1&scaling=scale-down&content-scaling=fixed&starting-point-node-id=230%3A1075> .